

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2022

Tập luận văn đội tuyển quốc gia Olympic Tin học

China Computer Federation, tháng 2 năm 2022

Nguồn gốc: Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2022

IOI China candidate team papers / National training team 2022 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2022. Tập này là bản quét ảnh: pdftotext chỉ trả về các dấu sang trang và không khôi phục được văn bản thân bài. Vì vậy bản LaTeX này chuyển ngữ phần nhận diện tập sách, mục lục và ba bài đầu tiên bằng cách đọc trực tiếp các trang đã render từ PDF, kết hợp OCR bằng tesseract.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2022*. Trang bìa ghi đơn vị China Computer Federation và thời điểm phát hành là tháng 2 năm 2022. Tập PDF gồm 246 trang; phần thân bắt đầu từ bài về duy trì thông tin đường đi trên một lớp DAG đặc biệt.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
1	Bàn về duy trì thông tin đường đi trên một lớp DAG đặc biệt	Dai Jiangqi
13	DFT tổng quát hơn	Zhou Hangrui
28	Bàn về ứng dụng của một tư tưởng tham lam dựa trên so sánh trong thi tin học	Zhang Junkai
49	Bàn về các thuật toán liên quan tới đồ thị phẳng	Tang Shaoxuan
72	Bàn về bài toán tương tác dạng tìm tham số tương tác	Hu Hao
82	Bàn về thuật toán xấp xỉ trong OI	Xu Tingqiang
99	Khảo sát bước đầu về bài toán tô màu đồ thị	Peng Bo
118	Bàn về bài toán tối ưu không gian trong thi tin học	Chen Zhixuan
129	Bàn về một lớp ma trận Monge	Zhang Jingxing
145	Bàn về phân tích độ phức tạp của một lớp bảng băm	Chen Yusi
159	Ứng dụng của đại số tuyến tính trong OI	Chang Ruinian
182	Bàn về các bài toán tổ hợp chuỗi liên quan tới lý thuyết Lyndon	Wan Chengzhang
204	Báo cáo ra đề <i>Chuỗi của Cutie Duo</i>	Feng Shiyuan
213	Bàn về bài toán mô phỏng luồng chi phí trong OI	Chen Kuowen

3 Bàn về duy trì thông tin đường đi trên một lớp DAG đặc biệt

Dai Jiangqi, Trường Ngoại ngữ Nam Kinh

Tóm tắt

Trong OI, không ít bài toán liên quan tới cấu trúc dữ liệu có thể được xem như bài toán đường đi trên DAG. Bài viết này giới thiệu ba thuật toán có thể duy trì thông tin đường đi trên DAG trong thời gian $O(n \text{ poly}(\log S))$, trong đó n là kích thước đầu vào, S là tổng số đường đi của DAG và $\text{poly}(x)$ biểu thị một đa thức theo x . Bài viết cũng khảo sát ứng dụng của các thuật toán này trong cấu trúc dữ liệu, chuỗi và số học.

Mở đầu

Trong OI, rất nhiều bài toán có thể quy về mô hình liên quan tới đường đi trên DAG, bao gồm nhưng không giới hạn ở đồ thị, cấu trúc dữ liệu và chuỗi. Trên DAG tổng quát, việc duy trì nhanh thông tin đường đi là không thực tế, vì chỉ riêng mô tả một đường đi đã cần $\Omega(n)$ bit. Do đó bài viết tập trung vào các DAG có tổng số đường đi tương đối nhỏ; nhiều mô hình DAG thực dụng, chẳng hạn máy tự động hữu tố, thỏa tính chất này.

Bài viết chủ yếu giới thiệu ba thuật toán có thể duy trì thông tin đường đi trên DAG trong thời gian $O(\text{polylog } S)$, với S là tổng số đường đi: phân rã chuỗi trên DAG, cây cân bằng bền vững và tách-trộn cây cân bằng. Trong đó, phân rã chuỗi trên DAG có cài đặt gọn, thao tác sửa đổi tương đối dễ, nên thường thực dụng hơn trong OI. Cây cân bằng bền vững có độ phức tạp thời gian tối ưu và khả năng mở rộng mạnh, nhưng tốn bộ nhớ và khó cài đặt hơn. Độ phức tạp thời gian của phương pháp tách-trộn cây cân bằng vẫn còn mở; hiện chỉ có chứng minh cận trên $O(\log S \log q)$ mà chưa xây dựng được dữ liệu làm chặt cận này. Tuy vậy, vì quan hệ giữa cập nhật và truy vấn của nó ngược với hai thuật toán trước, tức truy vấn là offline còn DAG có thể bị bắt buộc đưa ra online, nó vẫn có một số ứng dụng khó thay thế.

Phần 1 trình bày mô hình cơ bản của duy trì thông tin đường đi trên DAG. Phần 2 lần lượt trình bày ba thuật toán: phân rã chuỗi trên DAG, cây cân bằng bền vững và tách-trộn cây cân bằng. Phần 3 nêu các ứng dụng trong một số bối cảnh.

3.1 Mô hình cơ bản

Cho một DAG có n đỉnh, trong đó các cạnh ra của mỗi đỉnh có thứ tự. Sắp các đỉnh mà đỉnh i ($1 \leq i \leq n$) trở tới thành dãy G_i . Đỉnh i có trọng số nguyên dương c_i , và cạnh ra thứ j của nó có trọng số cạnh j .

Sắp tất cả các đường đi bắt đầu tại 1 và kết thúc ở một đỉnh có bậc ra bằng 0 theo thứ tự từ điển của dãy trọng số cạnh; cách sắp này là duy nhất. Ký hiệu dãy đường đi là P_i ($1 \leq i \leq S$), trong đó S là tổng số đường đi thỏa điều kiện.

Có q truy vấn. Mỗi truy vấn cho một số nguyên dương k , yêu cầu tính tổng trọng số đỉnh trên P_k .

Giả sử số cạnh của DAG không vượt quá $2n$. Dòng ràng buộc trong bản quét bị OCR nhiễu; các phần đọc được gồm $k \leq 10^{18}$ và các chặn đa thức thông thường cho n, q, c_i .

3.2 Lời giải

Vì $k \leq 10^{18}$, chỉ cần giữ lại 10^{18} đường đi đầu của DAG. Trên thực tế, có thể dùng DP $O(n)$ để tính số đường đi từ mỗi đỉnh tới các đỉnh bậc ra 0, đồng thời lấy min với 10^{18} , rồi chỉ giữ các đỉnh có giá trị DP nhỏ hơn 10^{18} . Ngoài ra cần sao chép các đỉnh nằm trên đường đi nhỏ thứ 10^{18} theo thứ tự từ điển và giữ với các cạnh ra của chúng một tiên tố trở tới nhóm đỉnh trước. Vì vậy chỉ cần xét trường hợp $S \leq 10^{18}$.

Bằng cách nhị phân hóa bậc ra, dễ thấy ta chỉ cần xét trường hợp bậc ra của mỗi đỉnh là 0 hoặc 2. Trong phân tích dưới đây giả sử $S > n$.

3.2.1 Phân rã chuỗi trên DAG

Nếu DAG là một cây có hướng, hoặc một rừng có hướng, tức bậc vào của mỗi đỉnh không vượt quá 1, hiển nhiên có thể dùng phân rã nặng–nhẹ để duy trì thông tin trên chuỗi.

Ta mở rộng cách làm này lên DAG tổng quát hơn. Với mỗi cạnh $u \rightarrow v$, xét số đường đi và số tiền tố đường đi liên quan tới hai đầu cạnh, rồi chọn cạnh nặng theo đỉnh kế tiếp và tiền nhiệm có giá trị lớn nhất. Bản quét nhận diện nhiều ở công thức định nghĩa chính xác của f_i, h_i, pre_i ; phần đọc được cho thấy f_i là số đường đi kết thúc ở i , h_i là số đường đi bắt đầu từ i , còn pre_i là tiền nhiệm nặng của i .

Cạnh $u \rightarrow v$ là cạnh nặng khi v là lựa chọn nặng của u và u là tiền nhiệm nặng của v . Dễ thấy các cạnh nặng tạo thành một số chuỗi rời nhau, gọi là chuỗi nặng.

Xét một đường đi bất kỳ trên DAG. Dọc đường đi đó, f_i giảm dần còn h_i tăng dần. Với mỗi cạnh nhẹ, ít nhất một trong hai đại lượng phải giảm hoặc tăng theo hệ số 2. Vì vậy trên một đường đi chỉ có $O(\log S)$ cạnh nhẹ.

Do đó, từ đỉnh 1 ta có thể mỗi lần tìm bằng nhị phân phần chung giữa đường đi được hỏi và chuỗi nặng hiện tại; trên mỗi chuỗi nặng tiền xử lý tổng tiền tố. Nhờ vậy bài toán ở trên được giải trong thời gian $O(n + q \log S \log n)$ và bộ nhớ $O(n)$. Nút thắt nằm ở phép nhị phân trong mảng f .

Tối ưu. Tham khảo kỹ thuật cây nhị phân cân bằng toàn cục trong phân rã cây, độ phức tạp có thể hạ xuống $O(n + q \log S)$. Với một chuỗi nặng cố định, đặt $w_i = f_i - f_{i+1}$, với quy ước $f_{s+1} = 0$. Dựng một cây nhị phân trên toàn bộ chuỗi sao cho trong một đoạn $[l, r]$, gốc con là vị trí đầu tiên có tổng trọng số hai phía đều không vượt quá một nửa tổng đoạn; nếu không tồn tại thì lấy đầu đoạn. Tiền xử lý LCA trên cây nhị phân này.

Nếu u là vị trí vào chuỗi nặng hiện tại và v là vị trí ra, ta tiền xử lý LCA giữa mỗi đỉnh với cuối chuỗi nặng rồi nhị phân từ $\text{LCA}(u, k)$ đi xuống. Độ phức tạp của phép nhị phân trên cây nhị phân chính là khoảng cách giữa $\text{LCA}(u, k)$ và v trên cây đó.

Bắt đầu từ v và đi lên hai bước mỗi lần, tổng trọng số của cây con ít nhất tăng gấp đôi, trừ tình huống lá biên. Sau một bước đi xuống từ LCA, tổng cây con không vượt quá đại lượng tương ứng ở điểm vào. Cộng trên mọi chuỗi nặng của một truy vấn, vì đại lượng của chuỗi hiện tại không nhỏ hơn chuỗi kế tiếp, tổng độ phức tạp nhị phân là $O(\log S)$.

Dùng splay hoặc treap cũng đạt cùng bậc độ phức tạp; bài viết không nhắc lại.

Mở rộng tối ưu. Dùng chia để trị, tức thông tin nửa nhóm trên đoạn với tiền xử lý $O(n \log n)$ và truy vấn $O(1)$ dựa trên hợp nhất hai đoạn, kết hợp DP, có thể hỗ trợ truy vấn giá trị lớn nhất của tổng trọng số đỉnh trên đoạn $P_l, P_{l+1}, \dots, P_r$ trong thời gian $O(n \log n + q \log S)$. Cụ thể, với mỗi đỉnh tính giá trị đường đi lớn nhất bắt đầu ở nó, rồi trên mỗi chuỗi nặng lưu cực trị đoạn của các thông tin đó.

Với các bài toán có sửa điểm đơn, sửa chuỗi đơn giản và truy vấn thông tin trên một chuỗi, cũng có thể tổng quát hóa cây đoạn có trọng số để giảm độ phức tạp mỗi thao tác xuống $O(\log S)$.

Bản quét chứa một công thức chọn gốc cây nhị phân phụ thuộc vào hai dãy $w_i = f_i - f_{i+1}$ và $x_i = h_i - h_{i-1}$, cùng tính chẵn lẻ của độ sâu d ; công thức này quá nhiều để chép chính xác. Ý chính đọc được là khoảng cách từ điểm vào tới LCA và từ điểm ra tới LCA đều bị chặn bởi các logarit của tỉ lệ tổng trọng số, nên cộng trên mọi chuỗi nặng vẫn không vượt quá $O(\log S)$.

Ngoài ra, với thông tin khả nghịch có sửa điểm đơn, xây cây đoạn có trọng số riêng theo hai phía cũng đạt truy vấn đơn $O(\log S)$. Tuy nhiên hằng số lớn; nếu thông tin cần duy trì đơn giản thì thuật toán này không có ưu thế rõ rệt so với BIT.

3.2.2 Cây cân bằng bền vững

Xét bài toán gốc theo thứ tự tô-pô đảo của DAG. Với mỗi đỉnh i , xét dãy Q_i gồm mọi đường đi bắt đầu từ i .

Rõ ràng, nếu bậc ra của i bằng 0 thì $Q_i = ((i))$. Nếu bậc ra của i bằng 2, với hai cạnh ra theo thứ tự, Q_i nhận được bằng cách ghép hai dãy đường đi của hai con, rồi chèn i vào đầu mỗi phần tử.

Ta chỉ duy trì tổng trọng số đỉnh của từng đường đi trong Q_i . Khi đó cần hỗ trợ các thao tác sau trong $O(\log S)$:

1. ghép hai dãy số nguyên độ dài không vượt quá S thành một dãy mới;
2. cộng cùng một hằng số vào mọi phần tử của một dãy;
3. cho k , hỏi phần tử thứ k của một dãy.

Dùng cây cân bằng để duy trì. Do có thao tác tự sao chép, ta cần một loại cây không có con trở cha và chiều cao chặt $O(\log S)$. Bài viết dùng WBLT, tức Weight Balanced Leafy Tree. Trong WBLT, mỗi lá biểu diễn một phần tử; mỗi nút trong có cây con trái và phải, và kích thước mỗi cây con không nhỏ hơn một hằng số cân bằng nhân với tổng kích thước. Mỗi nút có thể lưu thông tin đoạn. Ở đây, lá biểu diễn đường đi và mỗi nút giữ một nhãn cộng lười cho cây con.

Với thao tác ghép, chỉ cần merge hai WBLT. Khi ghép hai cây A, B và giả sử $|A| \geq |B|$, nếu tỉ lệ kích thước đã cân bằng thì tạo một gốc mới với hai cây con A, B . Nếu không, xét cây con phải của A ; khi ghép trực tiếp vẫn thỏa cân bằng thì gắn B vào phía phải. Nếu vẫn chưa thỏa, tiếp tục tách cấu trúc bên phải của A , ghép các phần tương ứng rồi hợp nhất trở lại. Bằng quy nạp, ghép hai WBLT có tỉ lệ kích thước là hằng số tổn $O(1)$; vì các đệ quy còn lại chỉ đi về một phía, tổng thời gian ghép không vượt quá $O(\log S)$. Trong bài này còn có thể lợi dụng tính khả trừ và giao hoán của thông tin để làm nhãn gần như vĩnh viễn, giảm hằng số số lần *pushdown*.

Thao tác cộng toàn dãy chỉ sửa nhãn ở gốc. Thao tác hỏi phần tử thứ k đệ quy từ gốc xuống lá. Vì vậy bài toán ban đầu được giải trong thời gian $O(n + q \log S)$ và bộ nhớ $O(n \log S)$.

Mở rộng của lời giải WBLT. Ngoài ghép, WBLT và một số cây cân bằng khác còn hỗ trợ tách: cho cây A và $k \in [0, |A|]$, trả về hai WBLT lần lượt biểu diễn k phần tử đầu và $|A| - k$ phần tử sau.

Cụ thể, đệ quy từ trên xuống sẽ tách A thành một số cây con có độ sâu tăng dần và một số cây con có độ sâu giảm dần; sau đó ghép chúng từ dưới lên để thu được hai cây kết quả. Vì chi phí ghép tỉ lệ với logarit tỉ lệ kích thước của hai phần, để chứng minh thao tác tách tổn $O(\log S)$.

Nói cách khác, ngoài ghép dãy, sửa toàn cục và hỏi điểm đơn, cây cân bằng bền vững còn hỗ trợ cắt lấy một đoạn dãy. Tất nhiên cũng có thể hỗ trợ sửa đoạn và hỏi đoạn; trong bài toán gốc, đoạn này tương ứng với một đoạn của dãy đường đi P .

Do cấu trúc phức tạp, cây cân bằng bền vững khó hỗ trợ sửa trọng số đỉnh của DAG.

Thảo luận liên quan. Có thể thấy mỗi DAG mà mọi đỉnh có bậc ra 0 hoặc 2 tương đương về cấu trúc với một cây Leafy Tree bền vững.

Điều này có nghĩa: khi không có tính chất đặc biệt và số thao tác ít, ta có thể duy trì trực tiếp DAG để cài ghép dãy và cắt dãy, bỏ qua các thao tác giữ cân bằng. Định nghĩa độ sâu dep_v của đỉnh v là số cạnh trên đường đi dài nhất bắt đầu ở v . Khi ghép A, B thành C , chỉ cần tạo một đỉnh mới có hai con A, B và đặt $\text{dep}_C = \max(\text{dep}_A, \text{dep}_B) + 1$. Khi tách A thành B, C , thời gian không vượt quá dep_A , vì trong quá trình đệ quy mỗi độ sâu chỉ bị truy cập ở trước và sau nhiều nhất một lần. Bằng cách ghép từ sau ra trước dọc ranh giới giữa B và C , ta đảm bảo $\max(\text{dep}_B, \text{dep}_C) \leq \text{dep}_A$. Sau $O(q)$ thao tác ghép và tách, độ sâu không vượt quá q , nên tổng độ phức tạp là $O(q^2)$. Đây cũng đạt cận dưới lý thuyết trong mô hình này, vì

sau q bước số đường đi có thể đạt $\exp(\Theta(q))$. Ưu thế của cây cân bằng bền vững nằm ở các trường hợp DAG có cấu trúc đặc biệt hơn.

Ngoài ra, cấu trúc phân rã chuỗi trên DAG kết hợp cây đoạn có trọng số ở phần trước cũng có thể được xem như một DAG “cân bằng hơn” nhưng tương đương với DAG gốc theo ý nghĩa dãy đường đi.

3.2.3 Tách-trộn cây cân bằng

Xét phiên bản offline của bài toán gốc: tại đỉnh 1, duy trì tập các chỉ số k xuất hiện trong mọi truy vấn.

Duyệt các đỉnh theo thứ tự tô-pô từ trước ra sau. Khi tới đỉnh i , mọi truy vấn đang nằm tại i được cộng thêm c_i vào đáp án. Nếu i có bậc ra 2, chuyển toàn bộ truy vấn này tới các đỉnh mà i trở tới; nửa sau cần trừ đi kích thước của nửa trước khỏi chỉ số k tương ứng.

Cần duy trì nhiều dãy không giảm và hỗ trợ ba thao tác sau; sau khi bị thao tác, dãy cũ biến mất:

1. tách một dãy thành hai dãy mới theo ngưỡng k ;
2. cộng k vào mọi phần tử của một dãy;
3. trộn hai dãy đã sắp xếp thành một dãy mới.

Dùng cây cân bằng để duy trì các dãy này thì hai thao tác đầu là trực tiếp.

Với thao tác trộn hai dãy A và B thành C , chia các vị trí $1, 2, \dots, |A| + |B|$ thành t đoạn liên tiếp cực đại sao cho các phần tử trong mỗi đoạn của C trước khi trộn đều thuộc cùng một dãy cũ. Dùng nhị phân từ trái sang phải hoặc đệ quy từ gốc xuống đều cho lời giải đơn giản $O(t \log q)$.

Tiếp theo chứng minh tổng t không vượt quá $O((n + q) \log S)$. Với một dãy A , định nghĩa thế năng

$$\Phi(A) = \sum_i \log(2 + A_{i+1} - A_i).$$

Tổng thế năng luôn không âm và không vượt quá $O(q \log S)$. Thao tác tách làm thế năng thay đổi nhiều nhất $O(\log S)$, thao tác cộng toàn dãy không đổi thế năng. Vì vậy chỉ cần chứng minh trong một thao tác trộn, thế năng giảm ít nhất $\Omega(t) - O(\log S)$.

Bản quét của chuỗi bất đẳng thức chi tiết bị nhiễu ở chỉ số, nhưng lập luận đọc được như sau: gọi các đoạn cực đại sau trộn là $[l_i, r_i]$ và đặt $d_i = l_i - r_{i-1}$ giữa hai đoạn kề nhau. Khi ghép hai đoạn kề, số hạng logarit tương ứng được thay bằng logarit của khoảng cách lớn hơn; dùng $\min(d_i, d_{i+1})$ và $\max(d_i, d_{i+1})$ để chặn, mỗi lần đổi đoạn đóng góp một lượng giảm hằng số, còn hai biên chỉ tạo sai số $O(\log S)$. Do đó tổng số đoạn cực đại trên mọi lần trộn bị chặn bởi tổng thế năng tăng.

Ta thu được lời giải thời gian $O((n + q) \log S \log q)$ và bộ nhớ $O(n + q)$.

3.2.4 Mở rộng và thảo luận

Nhìn lại bốn thao tác mà cây cân bằng bền vững ở phần trên hỗ trợ: ghép dãy, cắt dãy, cộng toàn cục và hỏi điểm đơn. Nếu xem các thao tác này như một DAG, rồi duyệt ngược và dùng cây cân bằng để duy trì mọi truy vấn, nút thất vẫn là tách cây, cộng toàn cục và trộn cây. Nói cách khác, vấn đề mà cây cân bằng bền vững và vấn đề mà tách-trộn cây cân bằng giải quyết phần nào là chuyển vị của nhau.

Đáng tiếc là cận trên tốt nhất đã biết của tách-trộn cây cân bằng là $O(\log S \log q)$, dù cũng chưa có dữ liệu đạt chặt cận này, còn cây cân bằng bền vững đạt $O(\log S)$. Trong các bài toán liên quan tới bài viết này, thuật toán tách-trộn chỉ hơn cây cân bằng bền vững khi có yêu cầu đặc biệt về bộ nhớ, hoặc khi một số thông tin trong DAG thao tác bị bắt buộc đưa ra online, chẳng hạn khi thao tác tách không phải tách theo “ $\geq k$ ” và “ $< k$ ” mà là tách theo k phần tử đầu và phần còn lại.

3.3 Ứng dụng

3.3.1 Duy trì quá trình thao tác trên dãy

Theo phần 2.2.2, xem DAG như một Leafy Tree bền vững giúp cài đặt thuận tiện các thao tác cắt dãy và ghép dãy.

Ví dụ 1. Có n dãy. Ban đầu dãy thứ i chỉ chứa một số.

Cần thực hiện q thao tác, mỗi thao tác thuộc một trong ba loại:

1. tạo một dãy mới bằng cách ghép hai dãy đã có;
2. tạo hai dãy mới, lần lượt là k phần tử đầu của một dãy đã có và các phần tử còn lại, trong đó k là số nguyên lớn không vượt quá độ dài dãy;
3. xuất số nghịch thế của một dãy modulo 998244353.

Ràng buộc đọc được: $1 < n < 100, 1 < q < 600$.

Lời giải ví dụ 1. Nếu không có thao tác loại 2, hiển nhiên có thể duy trì số lần xuất hiện của mỗi giá trị trong từng dãy và tổng số nghịch thế, mỗi thao tác tốn $O(n)$.

Dùng cách làm ở phần 2.2.2: với thao tác loại 2, ta đệ quy từ trên xuống để tách rồi ghép dần từ dưới lên, trong quá trình duy trì kích thước mỗi nút. Khi đó mỗi thao tác tách có thể viết lại thành $O(q)$ thao tác ghép, còn mỗi thao tác ghép tốn $O(n^2 + n\omega)$, trong đó ω là số bit của kích thước, thường xem là 32 hoặc 64. Tổng độ phức tạp là $O(n^2q^2 + n\omega q^2)$.

3.3.2 Ứng dụng trên máy tự động hậu tố

Trong lý thuyết chuỗi của OI, máy tự động hậu tố (SAM) là một thuật toán hiệu quả và thực dụng.

Cụ thể, với chuỗi S trên bảng chữ cái Σ , xét tập vị trí kết thúc của xâu con T trong S , ký hiệu là R_T . Với các T khác nhau, quan hệ giữa các R_T chỉ có thể là bao hàm hoặc rời nhau; vì vậy chỉ có $O(|S|)$ tập R_T khác nhau về bản chất và chúng tạo thành một cấu trúc cây, thực chất là cây hậu tố của chuỗi đảo sau khi nén các đỉnh bậc hai.

Định nghĩa tự động A có tập đỉnh là các R_T , với T là xâu con của S , và có cạnh ký tự a từ R_T tới R_{Ta} nếu Ta cũng là xâu con của S . Từ các tính chất của SAM, A có kích thước $O(|S|)$ và có thể xây dựng cây cùng tự động trong $O(|S|)$.

Các xâu con khác nhau của S tương ứng một-một với các đường đi bắt đầu từ xâu rỗng trong A . Nhờ đó, thông tin về xâu con khác nhau được chuyển thành thông tin đường đi trên tự động, giúp xử lý hiệu quả một số bài toán khá khó nếu chỉ nhìn từ góc độ cây hậu tố.

Ví dụ 2. Nguồn bài là UOJ Long Round #1, bài D¹, đã lược bỏ chi tiết không cần thiết.

Cho chuỗi S và mảng c_i . Với xâu con T của S , ký hiệu tập vị trí kết thúc của T trong S là R_T và định nghĩa $f(T) = |R_T| \sum_{i \in R_T} c_i$.

Có q truy vấn, mỗi truy vấn yêu cầu tính tổng $f(T)$ trên một họ xâu con liên tiếp theo mô tả của đề gốc. Ràng buộc đọc được: $|S| \leq 5 \cdot 10^5$, bảng chữ cái 26. Một phần dòng truy vấn trong bản quét bị OCR nhiễu nên không chép lại chính xác.

¹<https://uoj.ac/problem/577>

Lời giải ví dụ 2. Xây cây hậu tố của chuỗi đảo và SAM của S . Trên mỗi đỉnh của SAM, giá trị f là cố định, còn truy vấn đúng là tính tổng f trên một đường đi của SAM.

Dùng phân rã chuỗi trên DAG và duy trì tổng tiền tố trên mỗi chuỗi nặng là giải được. Nút thắt là tìm độ dài phần chung giữa đường đi truy vấn và chuỗi nặng hiện tại; dùng bảng thưa LCP với tiền xử lý $O(|S| \log |S|)$ và truy vấn $O(1)$.

Thảo luận ví dụ 2. Thực ra SAM chỉ có $O(|S|)$ đường đi tối đại, tương ứng với mọi hậu tố của S . Dựa trên tính chất này, tồn tại thuật toán đơn giản hơn.

Khi trải mọi đường đi của SAM ra, ta thu được một cây chỉ có $O(|S|)$ lá, thực chất là cây hậu tố của chuỗi thuận. Mỗi truy vấn tương đương tính tổng trọng số đỉnh trên một chuỗi của cây này; mặt khác các đỉnh này đều tương ứng với đỉnh SAM.

Nén các đỉnh bậc hai của cây này và xét thao tác nén trên tự động: xóa mọi đỉnh có bậc ra 1 trên tự động, với mọi cạnh trở tới chúng thì đổi đích cạnh thành đỉnh mà đích cũ trở tới, lặp đến khi đích có bậc ra khác 1, đồng thời đổi ký tự trên cạnh thành một chuỗi. Bằng cách lưu mảng trên cạnh, tự động sau nén vẫn duy trì được tổng đoạn; khi trải tự động ra sẽ trực tiếp thu được một cây hậu tố chuỗi thuận đã nén có $O(|S|)$ đỉnh. Xử lý tổng tiền tố trực tiếp trên cây này là đủ. Tổng độ phức tạp là $O(|S| + q)$.

Đáng chú ý, tập đỉnh của SAM nén của chuỗi thuận và chuỗi đảo là giống nhau về bản chất. Điều này đem lại một góc nhìn mới cho các bài toán xâu con liên quan tới cả đầu trái và đầu phải. Tuy nhiên thuật toán này cũng có hạn chế: cấu trúc SAM nén phụ thuộc vào tính đối xứng giữa chuỗi thuận và chuỗi đảo, nên khả năng mở rộng kém hơn phân rã chuỗi trên DAG.

Ví dụ 3. Cho một trie S có n đỉnh. Định nghĩa xâu con của S là chuỗi tương ứng với một đường đi từ trên xuống.

Có q truy vấn. Mỗi truy vấn cho k , yêu cầu tìm xâu con khác nhau nhỏ thứ k của S hoặc báo không tồn tại. Để tránh đầu ra quá lớn, chỉ cần xuất tổng mã ASCII của mọi ký tự trong xâu đó.

Ràng buộc đọc được: $n, q \leq 10^5$, $k \leq 10^{18}$, bảng chữ cái 26.

Lời giải ví dụ 3. Cần xây SAM tổng quát. Ở đây S được định nghĩa là trie tạo bởi các chuỗi trên đường đi từ mọi đỉnh về gốc, còn tập *right* của SAM tương ứng một-một với đỉnh của trie.

Phân tích độ phức tạp của thuật toán xây SAM trên chuỗi không còn áp dụng trực tiếp, và số cạnh của SAM lúc này có thể đạt $n|\Sigma|$. Theo các tài liệu được trích dẫn, BFS trực tiếp trên trie rồi dùng thuật toán SAM cho chuỗi là tuyến tính theo kích thước kết quả, nhưng tác giả không biết chứng minh đầy đủ. Cũng có thể dùng suffix array tổng quát bằng nhân đôi để tính cây, rồi từ đó tính SAM, tổng độ phức tạp $O(n \log n + n|\Sigma|)$; nếu dùng mở rộng của DC3 trên cây có thể bỏ thừa số log.

Sau khi xây được SAM, bài toán trở thành mô hình kinh điển của phần 1, và cả ba thuật toán ở phần 2 đều giải được. Chẳng hạn dùng phân rã chuỗi trên DAG thì tổng thời gian là $O(n \log n + n|\Sigma| + q \log^2 n)$, hoặc với tối ưu là $O(n \log n + n|\Sigma| + q \log n)$.

Ví dụ 4. Nguồn bài là UOJ tuyển chọn đội, bài C^2 .

Cho chuỗi S độ dài n và m xâu con của S làm mẫu. Có q truy vấn; mỗi truy vấn yêu cầu tổng số lần xuất hiện của mọi xâu mẫu trong một xâu con của S . Ràng buộc đọc được: $n \leq 4 \cdot 10^5$, $m \leq 10^5$, $q \leq 10^5$, bảng chữ cái 26.

²<https://uoj.ac/problem/697>

Lời giải ví dụ 4. Xây SAM của S và cây hậu tố của chuỗi đảo, rồi phân rã chuỗi trên DAG của SAM.

Với một xâu con T của S , số xâu mẫu trong các hậu tố của T được quyết định bởi đỉnh tương ứng với T trên SAM và các xâu mẫu tương ứng. Hơn nữa, chỉ cần lấy số xâu mẫu trên đường đi từ đỉnh đó về gốc trong cây hậu tố đảo, rồi trừ đi số xâu mẫu có độ dài lớn hơn $|T|$.

Với mỗi truy vấn, định vị xâu con thành một đường đi trên tự động và tính tổng đóng góp của mọi đỉnh trên đường đi đó. Định vị đường đi có thể làm trong $O((n+q)\log n)$ bằng cách tiền xử lý $O(n\log n)$ trên chuỗi gốc và các chuỗi nặng của DAG rồi hỏi LCP xâu con trong $O(1)$.

Với thống kê đáp án: loại đóng góp thứ nhất chỉ cần lưu tổng tiền tố của w_v trên mỗi chuỗi nặng; loại thứ hai chuyển thành bài toán đếm điểm trong hình bình hành nghiêng 45° trên từng chuỗi nặng, sau đổi tọa độ là bài toán đếm điểm hai chiều thông thường. Tùy cách cài LCP và đếm điểm, tổng độ phức tạp là $O((n+m)\log n + q\log^2 n)$ hoặc $O((n+m+q)\log n)$.

3.3.3 Thuật toán Euclid vạn năng

Trên lưới có đường thẳng $y = \frac{a}{b}x$ với $a, b \in \mathbb{N}^+$. Xét một điểm động trên đường thẳng này di chuyển theo hướng $x+y$ tăng trên một đoạn. Mỗi khi đi qua một đường ngang, ghi ký hiệu A ; mỗi khi đi qua một đường dọc, ghi ký hiệu B ; nếu đi qua điểm lưới thì ghi A trước rồi B sau. Cuối cùng thu được một dãy trên $\{A, B\}$, ví dụ với $a=3, b=2$ thì dãy là $ABBAB$.

Thay A và B bằng hai phần tử của một nửa nhóm, tức thông tin ghép được trong $O(1)$, mục tiêu của thuật toán Euclid vạn năng là tính phần tử nửa nhóm ứng với toàn bộ dãy.

Trước hết giả sử đoạn di chuyển là từ $(0,0)$ tới (a,b) và $\gcd(a,b)=1$. Nếu $a > b$, đặt $k = \lfloor a/b \rfloor$. Dễ thấy sau mỗi A đều có k ký hiệu B . Thay mọi đoạn AB^k trong dãy gốc bằng một ký hiệu mới tương ứng; về mặt hình học, điều này tương đương tác động một phép biến đổi tọa độ lên mặt phẳng và đường thẳng, đưa bài toán về kích thước nhỏ hơn. Lặp quá trình $O(\log a)$ lần sẽ giải được bài toán.

Dùng DAG để mô tả thuật toán trên: ban đầu có hai đỉnh A, B . Mỗi lần tạo một đỉnh C , nối cạnh tới A và B theo phép thay thế tương ứng, rồi thay (a,b) bằng $(a-b, b)$ hoặc $(a, b-a)$ tùy bên lớn hơn, cho tới khi $a=b$.

Số cạnh của DAG có thể đạt độ phức tạp của phép trừ lặp, tức $O(a)$. Chỉ cần dùng nhân đôi để tối ưu các thao tác trừ liên tiếp là có thể giảm kích thước đồ thị xuống $O(\log(a+b))$.

Dãy AB do quá trình Euclid vạn năng sinh ra chính là dãy các đỉnh cuối của mọi đường đi khi trải DAG theo thứ tự DFS. Vì vậy phiên bản tổng quát của thuật toán Euclid vạn năng tương đương truy vấn đoạn trên Leafy Tree bền vững ứng với DAG này. Nhiều bài toán Euclid cổ điển, như tổng $\lfloor ai/b \rfloor$ hoặc $\min_i (ai \bmod b)$, có thể chuyển thành mô hình này; hơn nữa DAG này cho phép giải một số bài toán phức tạp hơn liên quan tới quá trình Euclid.

Ví dụ 5. Đây là bài mẫu thuật toán Euclid vạn năng trên LOJ³.

Cho n, m, l và hai ma trận vuông A, B kích thước không vượt quá 10. Cần tính biểu thức ma trận theo dãy Euclid do đề bài định nghĩa. Bản quét của công thức mục tiêu bị nhiễu nên không chép lại an toàn. Ràng buộc đọc được: $n, m, l \leq 10^{18}$.

Lời giải ví dụ 5. Xét dãy AB ứng với đường thẳng liên quan trong đề. Duy trì ma trận M , ban đầu $M = I$. Từ vị trí thứ hai của dãy trở đi, mỗi khi gặp A thì đặt $M = MB$; mỗi khi gặp B thì cộng M vào đáp án rồi đặt $M = AM$. Khi kết thúc, đáp án là giá trị cần tìm.

Tác động của một đoạn của dãy có thể mô tả bằng một bộ ma trận: nếu trước khi đi vào đoạn có

³<https://loj.ac/p/6440>

trạng thái M , sau đoạn đáp án được cộng thêm một biểu thức tuyến tính theo M và trạng thái mới cũng là một biến đổi tuyến tính của M . Vì vậy thông tin đoạn ghép được trong $O(1)$, và có thể áp dụng trực tiếp phiên bản truy vấn đoạn của thuật toán Euclid vạn năng.

Ví dụ 6. Nguồn bài là CTT 2021 Day 4 T3, đã lược bỏ chi tiết không cần thiết.

Xét đồ thị vô hướng n đỉnh, trong đó i nối với $(i + d) \bmod n$. Trọng số đỉnh lấy theo một dãy tuần hoàn độ dài m : $w_i = a_{i \bmod m}$.

Cần thực hiện q lần sửa điểm đơn trên trọng số, sau mỗi lần sửa tính kích thước tập độc lập có trọng số lớn nhất của đồ thị.

Ràng buộc đọc được: $d < n \leq 10^9$, $m \leq 2 \cdot 10^5$, $w_i \leq 400$, $q \leq 10^5$.

Lời giải ví dụ 6. Dễ thấy đồ thị gồm $\gcd(n, d)$ chu trình, và chỉ có một số loại chu trình khác nhau về bản chất. Sau khi xóa mọi đỉnh từng bị sửa ít nhất một lần, đồ thị còn lại gồm $O(q)$ chuỗi và một số loại chu trình. Chỉ cần tiền xử lý đáp án của mỗi chuỗi và mỗi loại chu trình rồi cộng lại; với mỗi chuỗi, dùng một ma trận 2×2 biểu diễn trạng thái chọn hoặc không chọn hai đầu để tính tập độc lập có trọng số lớn nhất, rồi duy trì cây đoạn trên các điểm quan trọng.

Xét thông tin trên một chu trình. Trên lưới, vẽ đoạn $y = \frac{d}{n}x$ với $x \in [0, n)$ và xây dãy cùng DAG tương ứng của thuật toán Euclid vạn năng. Đồng thời duy trì một giá trị x ; các giá trị x khác nhau tương ứng với các chu trình khác nhau. Duyệt dãy và duy trì ma trận 2×2 ; mỗi khi gặp một loại ký hiệu thì cập nhật x theo modulo m , đồng thời ghép thông tin tương ứng với w_x . Sau khi thao tác xong, ma trận nhận được là đáp án của chu trình; nếu chỉ thao tác trên một đoạn thì nhận được đáp án của chuỗi.

Thông tin đoạn của dãy có thể định nghĩa là m ma trận 2×2 , biểu diễn tác động khi giá trị ban đầu của x lần lượt là $0, 1, \dots, m - 1$, cùng một độ dời cho x sau khi đi qua đoạn. DP trên DAG Euclid tính được mọi thông tin đoạn trong $O(m \log n)$, từ đó lập tức có đáp án chu trình.

Đáp án chuỗi là truy vấn đoạn trên thông tin đường đi của DAG. Vì DAG này chỉ có $O(\log n)$ đỉnh, DFS trực tiếp từ trên xuống đạt $O(q \log n)$. Tổng độ phức tạp là $O((m + q) \log n)$.

Kết luận

Bài viết đã giới thiệu ba thuật toán: phân rã chuỗi trên DAG, cây cân bằng bền vững và tách-trộn cây cân bằng, đồng thời đưa ra ứng dụng của chúng trong một số bàiOI. Tác giả phỏng đoán phía sau mô hình duy trì đường đi trên DAG có thể còn những tính chất đẹp và sâu hơn; các mở rộng khác của ba thuật toán và mối liên hệ giữa chúng vẫn cần được khai thác thêm. Hy vọng bài viết có thể gợi mở hướng nghiên cứu tiếp theo cho độc giả quan tâm.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn huấn luyện viên Yang Yuliang của đội tuyển quốc gia đã hướng dẫn; cảm ơn cha mẹ, nhà trường, các thầy cô và bạn học đã bồi dưỡng, giảng dạy và hỗ trợ; cảm ơn các anh chị và bạn bè đã trao đổi, góp ý, truyền cảm hứng và đọc bản thảo; cảm ơn mọi nguồn tư liệu tham khảo; và cảm ơn độc giả đã dành thời gian đọc bài viết.

Tài liệu tham khảo

1. Zhang Zheyu. *Bàn về thuật toán chia để trị trên cây*. Luận văn đội tuyển quốc gia IOI 2019, 2019.

2. Wang Siqi. *Leafy Tree và cây cân bằng trọng số được cài đặt từ nó*. Luận văn đội tuyển quốc gia IOI 2018, 2018.
3. SF. *Bàn về máy tự động hậu tố nén*. Luận văn đội tuyển quốc gia IOI 2020, 2020.
4. Liu Yanyi. *Mở rộng máy tự động hậu tố trên trie*. Luận văn đội tuyển quốc gia IOI 2015, 2015.
5. J. A. Blumer. *Algorithms for the directed acyclic word graph and related structures*. Ph.D. Thesis, Denver University, 1985.
6. A. Blumer, J. Blumer, D. Haussler, R. M. McConnell, A. Ehrenfeucht. *Complete inverted files for efficient text retrieval and analysis*. J. ACM 34(3)(1987), pp. 578–595.
7. J. Karkkainen, P. Sanders, S. Burkhardt. *Linear work suffix array construction*. J. ACM 53(6)(2006), pp. 918–936.

4 DFT tổng quát hơn

Zhou Hangrui, Trường Văn Uyên thuộc Tập đoàn Giáo dục Trung học Học Quân Hàng Châu

Tóm tắt

Biến đổi Fourier rời rạc được dùng rất rộng rãi trong thi tin học. Bài viết này mở rộng DFT truyền thống, dùng độ phức tạp hơi cao hơn để thực hiện phép chập do một quan hệ hai ngôi tổng quát hơn định nghĩa.

4.1 Định nghĩa liên quan và quy ước

4.1.1 Phép chập và các phép toán cơ bản

Định nghĩa 1.1. Với một tập G và một quan hệ hai ngôi $\varphi : G \times G \rightarrow G$, cho hai ánh xạ $A, B : G \rightarrow \mathbb{C}$. Định nghĩa phép chập $C = A \times B$ bởi

$$C(v) = \sum_{\varphi(x,y)=v} A(x)B(y).$$

Tổng của hai ánh xạ $C = A + B$ thỏa

$$C(v) = A(v) + B(v).$$

Tích của một ánh xạ với một số phức $B = cA$ thỏa

$$B(v) = cA(v).$$

Bài viết chỉ xét trường hợp (G, φ) tạo thành một nửa nhóm hữu hạn. Với ánh xạ $A : G \rightarrow \mathbb{C}$, trong bài này A_x và $A(x)$ có cùng ý nghĩa.

4.1.2 DFT

Định nghĩa 1.2. Với dãy kích thước $(a_i)_{i=1}^k$, định nghĩa tập dãy ma trận tương ứng là R , thỏa:

- $R = \{(m_i)_{i=1}^k \mid m_i \in M_{a_i}\}$, trong đó M_n chứa mọi ma trận phức $n \times n$;
- $C = A + B$ thỏa $C_i = A_i + B_i$;

- $C = AB$ thỏa $C_i = A_i B_i$;
- $B = cA$ thỏa $B_i = cA_i$.

Ở đây $A, B, C \in R$ là các dãy ma trận có cùng dãy kích thước, còn c là một số phức.

Định nghĩa 1.3. Với một quan hệ chập, định nghĩa DFT tương ứng của nó là một biến đổi tuyến tính khả nghịch

$$\mathcal{F} : (G \rightarrow \mathbb{C}) \rightarrow R,$$

trong đó R là tập dãy ma trận do một dãy kích thước $(a_i)_{i=1}^k$ định nghĩa, đồng thời thỏa:

- $\mathcal{F}(A) \times \mathcal{F}(B) = \mathcal{F}(AB)$;
- $\mathcal{F}(A) + \mathcal{F}(B) = \mathcal{F}(A + B)$;
- $c\mathcal{F}(A) = \mathcal{F}(cA)$;
- $\sum_{i=1}^k a_i^2 = |G|$.

Có thể xem $\mathcal{F}$ là một đẳng cấu.

Với phép nhân dãy ma trận, ta làm được trong $O(\sum_{i=1}^k a_i^\omega)$, trong đó ω là số sao cho phép nhân ma trận $n \times n$ có độ phức tạp $O(n^\omega)$; hiện đã biết $\omega \leq 2.3728596$ [5].

Không gian $(G \rightarrow \mathbb{C})$ có một hệ cơ sở $(A_x)_{(x \in G)}$, trong đó A_x là ánh xạ

$$t \mapsto [t = x] = \begin{cases} 1, & t = x, \\ 0, & t \neq x. \end{cases}$$

Vì mọi phần tử đều phân rã được thành tổ hợp tuyến tính của cơ sở, ta chỉ cần quan tâm tới $\mathcal{F}(A_x)$. Đặt $\mathcal{F}_x = \mathcal{F}(A_x)$.

Bổ đề 1.1. Một quan hệ DFT $\mathcal{F}$ tương ứng một-một với họ $(\mathcal{F}_x)$ thỏa:

- $\mathcal{F}_x \times \mathcal{F}_y = \mathcal{F}_{\varphi(x,y)}$;
- các $\mathcal{F}_x$ tạo thành một hệ cơ sở của R .

Chứng minh. Với một quan hệ DFT $\mathcal{F}$, do tính đẳng cấu, họ $(\mathcal{F}_x)$ mà nó thu được hiển nhiên hợp lệ. Ngược lại, với một họ $(\mathcal{F}_x)$, có thể dựng

$$\mathcal{F}(A) = \sum_{x \in G} A(x) \mathcal{F}_x.$$

Đối với phép nhân:

$$\begin{aligned} \mathcal{F}(A) \times \mathcal{F}(B) &= \sum_{x \in G} \sum_{y \in G} A_x B_y \mathcal{F}_x \times \mathcal{F}_y \\ &= \sum_{v \in G} \sum_{\varphi(x,y)=v} A_x B_y \mathcal{F}_x \times \mathcal{F}_y \\ &= \sum_{v \in G} \mathcal{F}_v \sum_{\varphi(x,y)=v} A_x B_y \\ &= \mathcal{F}(A \times B). \end{aligned}$$

Phép cộng và nhân vô hướng hiển nhiên thỏa. Tính song ánh suy ra từ việc $(\mathcal{F}_x)$ tạo thành một hệ cơ sở. ■

Do đó, khi đã biết $(\mathcal{F}_x)$, DFT có thể xem như phép nhân một véc-tơ hàng với ma trận biến đổi, độ phức tạp $O(|G|^2)$. Nếu cần IDFT, tức biến đổi ngược của DFT, việc tìm ma trận nghịch đảo cần tiền xử lý $O(|G|^\omega)$.

4.2 DFT truyền thống và nhóm cyclic

4.2.1 Phép chập dãy

Với phép chập dãy thông thường, quan hệ hai ngôi tương ứng là $(\mathbb{Z}, +)$. Tuy nhiên khi thao tác thực tế, ta thường lấy 2^k sao cho 2^k vượt quá độ dài dãy n , rồi chuyển quan hệ hai ngôi thành $(\mathbb{Z}_{2^k}, +)$; ở đây $\mathbb{Z}_p$ biểu thị nhóm cyclic cấp p .

4.2.2 Nhóm cyclic

Bổ đề 2.1. Với nhóm cyclic $\mathbb{Z}_p$, một DFT tương ứng là

$$\mathcal{F}_i = \{[1], [\omega_p^i], [\omega_p^{2i}], \dots, [\omega_p^{(p-1)i}]\},$$

trong đó ω_p là một căn đơn vị nguyên thủy bậc p .

Độc giả có thể tự kiểm tra rằng cách dựng này thỏa các điều kiện trên.

4.2.3 FFT và biến đổi chirp-Z

Với nhóm cyclic $\mathbb{Z}_p$ thỏa $p = 2^k$, có thể dùng FFT để tối ưu, đưa DFT về $O(p \log p)$.

Với nhóm cyclic $\mathbb{Z}_p$ tổng quát hơn, có thể dùng biến đổi chirp-Z để chuyển về trường hợp trên, cũng đạt $O(p \log p)$. Cụ thể:

$$\begin{aligned} \mathcal{F}(A)_i &= \left[\sum_{j=0}^{p-1} A_j \omega_p^{ij} \right] \\ &= \left[\frac{1}{\omega_p^{\binom{i}{2}}} \sum_{j=0}^{p-1} \frac{A_j}{\omega_p^{\binom{j}{2}}} \omega_p^{(i+j)} \right]. \end{aligned}$$

Đây là dạng của một phép chập trù; sau khi đảo thứ tự có thể chuyển thành phép chập dãy, rồi chuyển tiếp thành bài toán trên $\mathbb{Z}_{2^k}$. Độ phức tạp là $O(p \log p)$.

Vì vậy với nhóm cyclic, ta đều có thể hoàn thành DFT trong $O(|G| \log |G|)$; độ phức tạp của phép nhân ma trận hiển nhiên là $O(|G|)$.

4.3 Tổng trực tiếp và DFT tương ứng

4.3.1 Tổng trực tiếp

Định nghĩa 3.1. Với hai quan hệ hai ngôi $(G_1, \times)$ và $(G_2, \times)$, định nghĩa tổng trực tiếp $(G, \times)$ của chúng bởi:

- $G = \{(x, y) \mid x \in G_1, y \in G_2\}$;
- $(x_1, y_1) \times (x_2, y_2) = (x_1 \times x_2, y_1 \times y_2)$.

4.3.2 Xây dựng DFT

Trước hết chứng minh một bổ đề.

Bổ đề 3.1. Với không gian tuyến tính hữu hạn chiều, ký hiệu tích tensor là $\otimes$. Đặt $V = V_1 \otimes V_2 = \text{span}\{x \mid x = a \otimes b, a \in V_1, b \in V_2\}$. Nếu một cơ sở của V_1 là $(a_i)_{i=1}^n$, một cơ sở của V_2 là $(b_j)_{j=1}^m$, thì một cơ sở của V là

$$c = \{x \mid x = a_i \otimes b_j, i \in [1, n], j \in [1, m]\}.$$

Chứng minh. Hiển nhiên c là một hệ sinh của V , nên chỉ cần chứng minh các phần tử của nó độc lập tuyến tính. Phản chứng, đặt $c_{(i-1)m+j} = a_i \otimes b_j$ và giả sử

$$\sum_{i=1}^n \sum_{j=1}^m c_{(i-1)m+j} k_{i,j} = 0$$

với không phải mọi $k_{i,j}$ đều bằng 0. Đặt $S_i = \sum_{j=1}^m k_{i,j} b_j$, ta có $\sum_{i=1}^n S_i \otimes a_i = 0$. Vì các a_i tạo thành một cơ sở, suy ra $\forall i, S_i = 0$. Vì các b_j tạo thành một cơ sở, nếu $S_i = 0$ thì $k_{i,j} = 0$ với mọi $j \in [1, m]$. Do đó mọi $k_{i,j}$ đều bằng 0, mâu thuẫn. ■

Định nghĩa 3.2. Với hai dãy ma trận R và R' có dãy kích thước lần lượt là $(a_i)_{i=1}^n$ và $(a'_i)_{i=1}^m$, định nghĩa tích tensor của chúng $R \otimes R'$ bởi

$$(R \otimes R')_{(i-1)m+j} = R_i \otimes R'_j.$$

Định lý 3.1. Nếu $G_1 \rightarrow \mathbb{C}$ và $G_2 \rightarrow \mathbb{C}$ đều có DFT, thì $G \rightarrow \mathbb{C}$ cũng có DFT.

Chứng minh. Giả sử hai DFT lần lượt là $\mathcal{A}, \mathcal{B}$, với dãy kích thước tương ứng $(a_i)_{i=1}^n$ và $(b_i)_{i=1}^m$. Giả sử DFT của $G \rightarrow \mathbb{C}$ là $\mathcal{F}$, dãy kích thước tương ứng là $(c_i)_{i=1}^{nm}$. Có thể dựng như sau:

- $c_{(i-1)m+j} = a_i b_j$, với $i \in [1, n], j \in [1, m]$;
- $\mathcal{F}_{(x,y),(i-1)m+j} = \mathcal{A}_{x,i} \otimes \mathcal{B}_{y,j}$, trong đó $\otimes$ là tích tensor.

Với phép cộng và nhân vô hướng, kết luận hiển nhiên.

Do tính chất của tích tensor $(x_1 \otimes y_1)(x_2 \otimes y_2) = (x_1 x_2) \otimes (y_1 y_2)$, trong đó x_1, x_2 là ma trận $n \times n$ và y_1, y_2 là ma trận $m \times m$, với phép nhân ta có:

$$\begin{aligned} \mathcal{F}_{(x_1, y_1), v} \times \mathcal{F}_{(x_2, y_2), v} &= (\mathcal{A}_{x_1, a} \otimes \mathcal{B}_{y_1, b})(\mathcal{A}_{x_2, a} \otimes \mathcal{B}_{y_2, b}) \\ &= (\mathcal{A}_{x_1, a} \mathcal{A}_{x_2, a}) \otimes (\mathcal{B}_{y_1, b} \mathcal{B}_{y_2, b}) \\ &= \mathcal{F}_{(x_1, y_1) \times (x_2, y_2), v}, \end{aligned}$$

trong đó $v = (a-1)m + b, a \in [1, n], b \in [1, m]$.

Theo Bổ đề 3.1, các $(\mathcal{F}_x)$ tạo thành một cơ sở. ■

Suy ra hệ quả sau.

Bổ đề 3.2. Nếu

$$G \rightarrow \mathbb{C} = (G_1 \times G_2 \times \cdots \times G_k) \rightarrow \mathbb{C}$$

và mọi $G_i \rightarrow \mathbb{C}$ ($i \in [1, k]$) đều có DFT, thì có thể dựng DFT của $G \rightarrow \mathbb{C}$.

4.3.3 Độ phức tạp tốt hơn

Bổ đề 3.3. Ma trận biến đổi ứng với G là tích tensor của ma trận biến đổi ứng với G_1 và ma trận biến đổi ứng với G_2 .

Điều này suy ra ngay từ quá trình dựng DFT của G .

Định lý 3.2. Với $G \rightarrow \mathbb{C} = (G_1 \times G_2) \rightarrow \mathbb{C}$, ánh xạ $a : G \rightarrow \mathbb{C}$, việc áp dụng DFT ứng với G lên a tương đương lần lượt áp dụng DFT ứng với G_1 và G_2 . Cụ thể:

- Với $x \in G_1$, đặt G'_x thỏa $G'_{x,y} = G_{(x,y)}$. Áp dụng DFT trên G_2 cho từng G'_x , ghi kết quả là H_x .
- Gọi tập dãy ma trận ứng với G_2 là R , dãy kích thước tương ứng là $(b_i)_{i=1}^k$. Với $i \in [1, k]$, đặt $H'_{i,x} = H_{x,i}$. Áp dụng DFT trên G_1 cho từng H'_i . Chú ý ánh xạ lúc này trở thành $G_1 \rightarrow M_{b_i}$, nhưng quá trình tương tự.

Chỉ cần xét tính chất của ma trận biến đổi là đủ.

Tương tự, có hệ quả:

Bổ đề 3.4. Nếu

$$G \rightarrow \mathbb{C} = (G_1 \times G_2 \times \dots \times G_k) \rightarrow \mathbb{C},$$

thì áp dụng DFT ứng với G lên a tương đương lần lượt áp dụng DFT ứng với G_i ($i \in [1, k]$), có thể cần chia lại G .

Bổ đề 3.5. Độ phức tạp của cách làm này là

$$T = O\left(|G| \sum_{i=1}^k \frac{T_i}{|G_i|}\right),$$

trong đó T_i biểu thị độ phức tạp DFT trên G_i .

Ta có các hệ quả:

- nếu $T_i = O(|G_i| \log |G_i|)$, thì $T = O(|G| \log |G|)$;
- nếu $T_i = O(|G_i|^2)$, thì $T = O(|G| \sum |G_i|)$;
- nếu mọi $|G_i|$ đều là hằng số, có thể xem $T = O(k|G|) = O(|G| \log |G|)$.

Ví dụ, FWT n chiều là tổng trực tiếp của n bản sao $\mathbb{Z}_2 \rightarrow \mathbb{C}$, độ phức tạp là $O(|G| \log |G|) = O(n2^n)$.

Ví dụ 1 (Numbers). Nguồn bài: Zhang Huaihuai, <https://ioihw21.duck-ac.cn/problem/101>.

Cho n, k và hai mảng độ dài k là m, p với $\sum_{i=1}^k p_i = 1$. Trạng thái là một k -bộ c , ban đầu mọi phần tử đều bằng 0. Mỗi lần thao tác, với xác suất p_i ta đặt $c_i := (c_i + 1) \bmod m_i$. Hãy tính kỳ vọng của số trạng thái khác nhau về bản chất đã đi qua trước lần đầu quay lại trạng thái ban đầu. $T = \prod m_i \leq 5 \cdot 10^5$. Dữ liệu được đảm bảo ngẫu nhiên, không cần xét các trường hợp biên.

Lời giải. Đặt $G = \mathbb{Z}_{m_1} \times \mathbb{Z}_{m_2} \times \dots \times \mathbb{Z}_{m_k}$, và 0 là trạng thái ban đầu.

Xét việc tính xác suất mỗi trạng thái được đi qua; dưới đây giả sử cần tính xác suất trạng thái $x \in G$ được đi qua. Giả định rằng khi đạt tới trạng thái ban đầu hoặc x , thao tác lập tức kết thúc. Gọi f_i là số lần kỳ vọng đi qua trạng thái i trước khi đạt tới trạng thái ban đầu hoặc x . Đặc biệt, cường bức đặt $f_0 = 1$, $f_x = 0$. Với trạng thái khác y , ta có

$$f_y = \sum_{i=1}^k f_{d(y,i)} p_i,$$

trong đó $d(y, i)$ biểu thị trạng thái thu được khi giảm phần tử thứ i của trạng thái y đi 1. Khử Gauss trực tiếp làm được trong $O(T^4)$.

Đưa vào biến d_0, d_x thỏa:

$$f_0 = \sum_{i=1}^k f_{d(0,i)} p_i - d_0 = 1, \quad f_x = \sum_{i=1}^k f_{d(x,i)} p_i - d_x = 0.$$

Kiểm tra cho thấy d_x chính là giá trị cần tìm.

Gọi F là ánh xạ $x \mapsto f_x$, và P là ánh xạ

$$x \mapsto \begin{cases} p_i, & x_j = 1 \iff i = j, \\ 0, & \text{ngược lại.} \end{cases}$$

Tương tự, với d_0, d_x định nghĩa $D : y \mapsto d_y$. Khi đó các phương trình trên có thể viết thành:

$$F = F \times P - D, \quad f_0 = 1, \quad f_x = 0.$$

Mặt khác, $F = F \times P - D$ tương đương

$$\mathcal{F}(F) = \mathcal{F}(F) \times \mathcal{F}(P) - \mathcal{F}(D),$$

tức tương đương T phương trình:

$$\mathcal{F}_i(P_i - 1) = \mathcal{D}_i \quad (i \in G).$$

Ở đây $\mathcal{F}_g (g_i \in G)$ và $\mathcal{F}(F)_i (i \in [1, |G|])$ tương ứng một-một; $\mathcal{P}, \mathcal{D}$ được định nghĩa tương tự.

Ta thấy $\mathcal{P}_0 = \sum p_i = 1$; thực tế, với nhóm hữu hạn, mọi kết quả DFT đều có cộng dồn. Phương trình tương ứng là $\mathcal{D}_0 = d_0 + d_x = 0$. Do dữ liệu ngẫu nhiên, xác suất để $\mathcal{P}_i (i \neq 0)$ bằng 0 là rất nhỏ.

Định nghĩa $t(a, b)$ bởi

$$\mathcal{F}(C)_b = \sum C_a t(a, b),$$

tức là hệ số của ma trận biến đổi. Khi đó $t(a, b) = t(b, a)$ và $t(a, b)^{-1} = t(a^{-1}, b)$. Ngoài ra, $f_0 = 1$ tương đương $\sum \mathcal{F}_i = |G|$, còn $f_x = 0$ tương đương $\sum \mathcal{F}_i / t(i, x) = 0$.

Lúc này hệ phương trình phức tạp ban đầu được chuyển thành dạng:

$$\sum \mathcal{F}_i = |G|, \tag{1}$$

$$\sum \frac{\mathcal{F}_i}{t(x, i)} = 0, \tag{2}$$

$$\mathcal{F}_i(\mathcal{P}_i - 1) = \mathcal{D}_i = d_x(t(x, i) - 1) \Rightarrow \mathcal{F}_i = \frac{t(x, i) - 1}{\mathcal{P}_i - 1} d_x \quad (i \in G \setminus \{0\}). \tag{3}$$

Thay phương trình (3) vào (1), (2) thu được:

$$\mathcal{F}_0 + \left(\sum \frac{1}{\mathcal{P}_i - 1} - \sum \frac{1}{t(x, i)(\mathcal{P}_i - 1)} \right) d_x = 0,$$

$$\mathcal{F}_0 = \left(\sum \frac{1}{t(x, i)(\mathcal{P}_i - 1)} - \sum \frac{1}{\mathcal{P}_i - 1} \right) d_x,$$

$$\mathcal{F}_0 + \sum \frac{t(x, i) - 1}{\mathcal{P}_i - 1} d_x = |G|,$$

$$\left(\sum \frac{1}{t(x, i)(\mathcal{P}_i - 1)} - \sum \frac{1}{\mathcal{P}_i - 1} \right) d_x + \sum \frac{t(x, i) - 1}{\mathcal{P}_i - 1} d_x = |G|,$$

$$\left(\sum \frac{1}{t(x, i)(\mathcal{P}_i - 1)} + \sum \frac{t(x, i)}{\mathcal{P}_i - 1} - 2 \sum \frac{1}{\mathcal{P}_i - 1} \right) d_x = |G|.$$

Tính trực tiếp theo công thức cuối cùng đạt $O(|G|^2)$.

Xét tối ưu. Nút thắt là ba tổng ở vế trái. Với ánh xạ $x \mapsto 1/(\mathcal{P}_x - 1)$, thực hiện DFT; phần thứ nhất và thứ hai lần lượt là phần tử thứ x và x^{-1} của kết quả. Phần thứ ba là một hằng số. Do đó ta tối ưu được độ phức tạp về độ phức tạp DFT, tức $O(T \log T)$, đủ để qua bài.

Đây là một ứng dụng kinh điển của DFT: dùng DFT chuyển hệ phương trình chập phức tạp thành các phương trình nhân điểm đơn giản, rồi dùng độ dời và giá trị đặc biệt để suy ra phương trình giải độ dời, cuối cùng giải được đáp án.

4.4 DFT trên nửa nhóm giao hoán hữu hạn

4.4.1 DFT trên nhóm giao hoán hữu hạn

Phân rã nhóm giao hoán hữu hạn.

Định nghĩa 4.1. Nếu một bộ phần tử $[a_1, a_2, \dots, a_m]$ của nhóm giao hoán hữu hạn G thỏa:

- $\forall x \in G, \exists (k_i)_{i=1}^m$ sao cho $\prod_{i=1}^m a_i^{k_i} = x$;
- $\forall (k_i)_{i=1}^m$, nếu $\prod_{i=1}^m a_i^{k_i} = e$ thì $a_i^{k_i} = e$ với mọi $i \in [1, m]$,

thì bộ phần tử này là một phân rã của G .

Bổ đề 4.1. $[a_1, a_2, \dots, a_m]$ là một phân rã của G khi và chỉ khi

$$(a_1) \times (a_2) \times \dots \times (a_m) = G.$$

Xét hai tính chất trên, chứng minh khá đơn giản.

Tiếp theo ta chứng minh mọi nhóm giao hoán hữu hạn đều tồn tại phân rã, đồng thời đưa ra cách dựng.

Bổ đề 4.2. Mọi nhóm giao hoán hữu hạn G đều có thể phân rã thành tổng trực tiếp của các nhóm p giao hoán hữu hạn.

Chứng minh. Gọi $o(x)$ là cấp của x . Giả sử cấp của G là $p^r m$, với p nguyên tố và $p \nmid m$. Đặt

$$G_p = \{x \mid o(x) \mid p^r\}, \quad G_m = \{x \mid o(x) \mid m\}.$$

Ta chứng minh $G_p \times G_m = G$.

Dựng ánh xạ $\varphi : (x, y) \mapsto xy$. Hiển nhiên đây là một đồng cấu nhóm; chỉ cần chứng minh nó toàn ánh và $\ker \varphi = \{e\}$.

Với phần tử bất kỳ x , theo định lý Lagrange, $o(x) \mid p^r m$. Đặt $o(x) = p^s m'$ với $p \nmid m'$, khi đó $o(x^{p^s}) = m' \mid m$ và $o(x^{m'}) = p^s \mid p^r$. Vì $(p^s, m) = 1$, tồn tại u, v sao cho $up^s + vm = 1$, nên $x = x^{up^s + vm} = x^{up^s} x^{vm}$. Do đó φ là toàn ánh.

Nếu $x \in G_p, y \in G_m$ và $xy = e$, vì $o(x) = o(x^{-1}) = o(y)$, ta có $o(x) \mid (p^r, m) = 1$, nên $x = y = e$. Vậy $\ker \varphi = \{e\}$.

Do đó φ là một đẳng cấu nhóm. Lặp lại quá trình này, ta có thể phân rã G thành tổng trực tiếp của một số nhóm p giao hoán hữu hạn. ■

Phân rã nhóm p giao hoán hữu hạn.

Bổ đề 4.3. Với nhóm p giao hoán hữu hạn G , lấy một phần tử a có cấp lớn nhất và đặt $H = \langle a \rangle$. Khi đó với mọi $xH \in G/H$, tồn tại $y \in xH$ sao cho $o(xH) = o(y)$.

Chứng minh. Đặt $o(a) = p^n$, $o(x) = p^t$, $o(xH) = p^s$; hiển nhiên $t \geq s \geq k$. Xét $x^{p^s} = (xH)^{p^s} = H$, nên $x^{p^s} \in H$. Đặt $x^{p^s} = a^r$, khi đó

$$p^{t-s} = o(x^{p^s}) = o(a^r) = \frac{o(a)}{(o(a), r)} = \frac{p^n}{(p^n, r)}.$$

Suy ra $(o(a), r) = p^{n-t+s}$. Không mất tính tổng quát, đặt $r = up^{n-t+s}$.

Đặt $y = xa^{p^{n-t}v} \in xH$, trong đó $v = -up^{n-t}$. Khi đó

$$(yH)^{p^s} = y^{p^s}H = x^{p^s}a^{p^s p^{n-t}v}H = H.$$

Từ đó suy ra có thể chọn đại diện trong lớp xH có cấp đúng bằng $o(xH)$. ■

Bổ đề 4.4. Mọi nhóm p giao hoán hữu hạn G đều có thể được phân rã.

Chứng minh. Quy nạp theo $|G| = p^n$, giả sử mọi nhóm G' có $|G'| < p^n$ đều tồn tại phân rã. Tìm phần tử có cấp lớn nhất $a \in G$, $o(a) = p^k$. Nếu $k = n$ thì $[a]$ chính là một phân rã.

Đặt $H = \langle a \rangle$ và $G' = G/H$. Theo giả thiết quy nạp, G' tồn tại phân rã; giả sử là $[a_1H, a_2H, \dots, a_mH]$. Theo Bổ đề 4.3, chọn các đại diện sao cho $o(a_i) = o(a_iH)$ với $i \in [1, m]$.

Ta chứng minh $[a, a_1, a_2, \dots, a_m]$ là một phân rã.

Với mọi $x \in G$, xét $xH \in G/H$. Có

$$xH = \prod_{i=1}^m (a_iH)^{k_i} = \left(\prod_{i=1}^m a_i^{k_i} \right) H.$$

Do đó tồn tại t sao cho $x = a^t \prod_{i=1}^m a_i^{k_i}$.

Ngược lại, nếu $a^t \prod_{i=1}^m a_i^{k_i} = e$, thì $\prod_{i=1}^m (a_iH)^{k_i} = H$. Theo tính chất của a_iH , suy ra $(a_iH)^{k_i} = H$ với mọi i , tức $a_i^{k_i} \in H$. Vì $o(a_i) = o(a_iH)$, ta có $a_i^{k_i} = e$. Khi đó cũng suy ra $a^t = e$. ■

Định lý 4.1. Mọi nhóm giao hoán hữu hạn đều có thể được phân rã.

Điều này suy ra ngay từ Bổ đề 4.2 và Bổ đề 4.4.

Một cách cài đặt đơn giản hơn. Chứng minh trên thực ra đã cho một quá trình dựng, nhưng vẫn quá phức tạp.

Định lý 4.2. Với nhóm giao hoán hữu hạn G , duy trì dãy B , ban đầu rỗng, và $S = \text{span}(B)$. Đặt $f(x) = \min\{k \mid x^k \in S\}$. Lập các thao tác:

- nếu $G = S$ thì kết thúc;
- tìm mọi $x \in G$, $x \notin S$, và chọn phần tử a có $o(x)/f(x)$ lớn nhất;
- $B := B + a$, $S = \text{span}(B)$.

Dãy B thu được là một phân rã.

Xét mệnh đề mạnh hơn sau.

Bổ đề 4.5. Với mọi $n \geq 0$, tại một thời điểm nào đó nếu $B = [a_1, a_2, \dots, a_n]$, thì tồn tại một phân rã $[a_1, a_2, \dots, a_m]$ ($m \geq n$) sao cho $o(a_i) = p_i^{k_i}$ với $n < i \leq m$, p_i nguyên tố và $k_i \in \mathbb{Z}^+$.

Chứng minh. Quy nạp theo n ; với $n = 0$ hiển nhiên đúng. Giả sử trước khi phần tử thứ n được thêm vào, một phân rã hợp lệ là $B' = [a_1, \dots, a_r]$.

Với phần tử x , nếu $x = \prod_i (a'_i)^{t_i}$, thì

$$f(x) = \text{lcm}_i \frac{o(a'_i)}{(o(a'_i), t_i)}.$$

Khi $f(x)$ lớn nhất, có thể tìm một lũy thừa p^k và một chỉ số i sao cho $(o(a'_i)/p^k, t_i) = 1$ và k đạt lớn nhất; từ đó dựng được một phân tử x có $o(x) = f(x)$, $x \notin S$ và $f(x) = \max_{y \in G \setminus S} f(y)$.

Dựng

$$B = (a_1, \dots, a_{n-1}, x) + R,$$

trong đó R nhận được bằng cách thay trong phân rã cũ một phần tử tương ứng bởi các phần còn lại. Không khó chứng minh $\text{span}(B)$ chứa phần tử bị xóa, nên $\text{span}(B) = G$; hơn nữa cấp của các phần tử trong B vẫn đúng dạng trên, vì vậy đây là một phân rã. ■

Bổ đề 4.6. Với cài đặt thích hợp, cách làm này có độ phức tạp $O(|G|^2)$.

Chứng minh. Nút thắt độ phức tạp nằm ở việc tính $f(x)$. Ở vòng thứ n , ta có

$$f(x) \leq \prod_i o(a_i), \quad f(x) < \frac{|G|}{\prod_i o(a_i)}.$$

Vì vậy tổng chi phí tính $f(x)$ không vượt quá $O(|G|)$ cho mỗi phần tử, dẫn tới tổng $O(|G|^2)$. ■

Nửa nhóm giao hoán hữu hạn.

Định lý 4.3. $G \rightarrow \mathbb{C}$ tồn tại DFT khi và chỉ khi G thỏa luật tuần hoàn.

Chứng minh và cách dựng được trình bày trong tài liệu [6], bài viết này không triển khai chi tiết.

4.5 Nhóm đối xứng

4.5.1 Định nghĩa

Định nghĩa 5.1. Định nghĩa tích của hai biến đổi tuyến tính là tác động liên tiếp của hai biến đổi tuyến tính, tức phép nhân ma trận. Khi đó, với không gian véc-tơ V n chiều, mọi biến đổi tuyến tính không suy biến, tức có định thức khác 0, tạo thành nhóm tuyến tính tổng quát n chiều, ký hiệu là $GL(V)$. Nhóm con $L(V)$ của nó được gọi là nhóm biến đổi tuyến tính trên V .

Định nghĩa 5.2. Với nhóm G , nếu tồn tại một đồng cấu p từ G tới nhóm biến đổi tuyến tính $L(V)$ trên không gian tuyến tính V , thì p được gọi là một biểu diễn tuyến tính của nhóm G . Không gian V là không gian biểu diễn; số chiều của V là số chiều của biểu diễn, ký hiệu là d_p .

Định nghĩa 5.3. Giả sử hai biểu diễn p, p' của nhóm G trên không gian V lần lượt lấy cơ sở

$$e = (e_1, e_2, \dots, e_n), \quad e' = (e'_1, e'_2, \dots, e'_n).$$

Nếu $e = e'X$, $\det X \neq 0$, và với mọi $g \in G$ có $p(g) = X^{-1}p'(g)X$, thì gọi p, p' là tương đương.

Định nghĩa 5.4. Nếu p là một biểu diễn của nhóm G trên không gian tuyến tính V , và không tồn tại không gian con $W \subset V$ thỏa $W \neq 0$, $W \neq V$ và với mọi $g \in G$, $x \in W$ đều có $p(g)x \in W$, thì p là một biểu diễn bất khả quy.

Ký hiệu tập mọi biểu diễn bất khả quy đôi một không tương đương của G là $R(G)$.

4.5.2 DFT của nhóm đối xứng

Tiếp theo bài viết đưa ra thuật toán DFT cho nhóm S_n . Do giới hạn dung lượng và trình độ của tác giả, phần này không đưa ra chứng minh và cài đặt cụ thể; xem các tài liệu [1], [2], [3].

Định lý 5.1 (định lý Burnside). Với nhóm hữu hạn G , ta có

$$\sum_{p \in R(G)} d_p^2 = |G|.$$

Định lý này chứng minh rằng sau DFT, lượng thông tin là đủ.

Bổ đề 5.1. Với nhóm hữu hạn G , cách dựng sau là một DFT hợp lệ:

- $a_i = d_{p_i}$;
- $\mathcal{F}(A)_i = \sum_{g \in G} A_g p_i(g)$.

Điều này cho thấy biểu diễn nhóm và DFT có liên hệ rất chặt. Nếu đã biết $R(G)$, ta có thể hoàn thành DFT trong $O(|G|^2)$. Đáng tiếc là hiện không có một thuật toán tổng quát để tìm $R(G)$ của nhóm hữu hạn tùy ý G .

Bổ đề 5.2. Với cách dựng trên, một cách tính IDFT khác là

$$A_x = \frac{1}{|G|} \sum_{p_i \in R(G)} d_{p_i} \text{Tr}(p_i(x^{-1}) \mathcal{F}(A)_i).$$

Bổ đề 5.3. $R(S_n)$ có quan hệ một-một với các phân hoạch của n .

Tiếp theo ta đưa ra một cách dựng gọi là biểu diễn bán chuẩn Young.

Định nghĩa 5.5. Với hai bảng Young chuẩn bậc n có cùng hình dạng T_1, T_2 , định nghĩa thứ tự không nhân của chúng như sau. Nếu số n nằm ở hai hàng khác nhau trong T_1, T_2 , thì $T_1 < T_2$ khi và chỉ khi hàng của n trong T_1 nhỏ hơn. Nếu không, đặt T'_1, T'_2 là hai bảng Young thu được sau khi xóa phần tử n khỏi T_1, T_2 , và đặt $T_1 < T_2$ khi và chỉ khi $T'_1 < T'_2$.

Định nghĩa 5.6. Với một bảng Young T , giả sử phần tử n, m lần lượt nằm ở các hàng x_1, x_2 và các cột y_1, y_2 . Định nghĩa khoảng cách trực

$$d(n, m) = x_2 - x_1 + y_1 - y_2,$$

chú ý không lấy trị tuyệt đối.

Bổ đề 5.4. Hình dạng của các bảng Young bậc n tương ứng một-một với các phân hoạch của n . Chỉ cần xét độ dài từng hàng của bảng Young là thấy.

Định nghĩa 5.7. Với phân hoạch a của n , sắp các bảng Young tương ứng theo thứ tự không nhân thành T . Ký hiệu khoảng cách trực của a, b trong T_i là $d_i(a, b)$. Dưới đây cho cách dựng $p((t, t + 1))$; đặt $p((t, t + 1)) = a$.

- Với mọi i , $\alpha_{i,i} = d_i(t, t + 1)^{-1}$.
- Nếu đổi chỗ t và $t + 1$ trong T_i vẫn là bảng Young chuẩn hợp lệ, gọi bảng thu được là T_j , thì

$$\alpha_{i,j} = \begin{cases} 1 - d_i(t, t + 1)^{-2}, & i < j, \\ 1, & i > j. \end{cases}$$

- Các vị trí còn lại đều đặt bằng 0.

Với hoán vị bất kỳ p , dùng sắp xếp nổi bọt có thể chứng minh nó phân rã được thành tích của một số hoán vị dạng $(i, i + 1)$; từ đó hợp nhất để thu được $p(p)$. Có thể chứng minh đây là một họ biểu diễn bất khả quy và đôi một không tương đương. Tính trực tiếp, chi phí DFT là $O(|G|^2)$.

4.5.3 Độ phức tạp tốt hơn

Xét S_n và đặt

$$H = \{x \mid x(n) = n\} \sim S_{n-1}.$$

Khi đó $\{s_i H \mid s_i \in S_n\}$ tạo thành một phân hoạch theo các lớp kề của S_n . Biến đổi công thức định nghĩa DFT:

$$\begin{aligned} \mathcal{F}(A) &= \sum_{g \in G} A_g p_i(g) \\ &= \sum_t p_i(s_t) \sum_{x \in H} A_{s_t x} p_i(x). \end{aligned}$$

Đặt $A'_x = A_{s_t x}$.

Định lý 5.2. Với $x \in H$, dạng của $p(x)$ là

$$\begin{bmatrix} p_{i_1}(x) & 0 & \cdots & 0 \\ 0 & p_{i_2}(x) & \cdots & 0 \\ 0 & 0 & \ddots & p_{i_k}(x) \end{bmatrix},$$

trong đó i_j biểu thị hình dạng phân hoạch hoặc bảng Young bậc $n - 1$ có thể thu được sau khi xóa một phần tử khỏi hình dạng phân hoạch hoặc bảng Young a_j .

Chỉ cần khảo sát vị trí của n trong bảng Young. Các bảng có cùng vị trí của n chắc chắn liên tiếp trong T . Vì $x(n) = n$, xóa n không ảnh hưởng quá trình dựng. Quy nạp là chứng minh được.

Do đó với $A' : S_n \rightarrow \mathbb{C}$, ta có thể giải riêng từng $\mathcal{F}(A')$, rồi đệ quy thành bài toán con $S_{n-1} \rightarrow \mathbb{C}$, sau đó hợp nhất bằng vài lần nhân dãy ma trận.

Bổ đề 5.5. Trong các ma trận $C_t = p((t, t + 1))$ với $t \in [1, n - 2]$ và $D_t = D_{\text{prev}} A_t p(t)$ với $t \in [1, n]$, tổng số phần tử khác 0 là $O(n!)$.

Chứng minh. Xét mỗi phân hoạch hoặc hình dạng bảng Young a của $n - 1$. Nó đóng góp nhiều nhất một lần vào mỗi C_t và D_t , tức xuất hiện nhiều nhất $2n - 2$ lần. Ta biết $\sum a_i = (n - 1)!$, nên tổng đóng góp không vượt quá $O(n!)$. ■

Với s_i , ta chọn $s_i = (n, n-1, \dots, i)$. Khi đó $s_i = s_{i-1} \times (i, i+1)$; như vậy chỉ cần n lần nhân dãy ma trận là tính được $p(s_i)$. Chú ý rằng nhờ Bổ đề 5.5, tổng độ phức tạp của phép cộng và phép nhân ma trận không vượt quá $O(n!n)$.

Vì vậy độ phức tạp DFT của S_n là

$$T(n) = nT(n-1) + O(n!n^2) = O(n!n^3),$$

tốt hơn rõ rệt so với cách thô bạo $O((n!)^2)$.

Thuật toán này dùng chia để trị tương tự FFT, đệ quy các trường hợp khác nhau về cùng một bài toán con để giảm độ phức tạp. Với các nhóm tổng quát hơn, nếu tìm được cách dựng tương tự, ta cũng có thể hoàn thành DFT với độ phức tạp thấp.

Kết luận

DFT đã được dùng rộng rãi trong OI, nhưng trong thời gian dài ứng dụng của DFT hầu như chỉ giới hạn ở phép chập dãy; những năm gần đây mới mở rộng tới chuỗi lũy thừa tập hợp. Tuy nhiên với các định nghĩa phép chập tổng quát hơn, cộng đồng OI chưa nghiên cứu nhiều. Bài viết này cho thấy nhóm không giao hoán cũng có thể thực hiện phép chập với độ phức tạp cao hơn, đồng thời chỉ ra quan hệ giữa DFT và biểu diễn nhóm. Tác giả chỉ trình bày DFT của nhóm đối xứng, chưa trình bày ứng dụng của nó; ý nghĩa sâu hơn và ứng dụng đa dạng hơn của DFT vẫn cần các bạn đọc quan tâm tiếp tục nghiên cứu.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn thầy Xu Xianyou đã quan tâm và hướng dẫn; cảm ơn bạn Xu Tingqiang và anh Ma Yaohua đã giúp đỡ về nội dung; cảm ơn bạn Xu Tingqiang, bạn Luo Kai, anh Jin Ce và anh Xu Yinzhan đã đọc kiểm tra bản thảo.

Tài liệu tham khảo

1. Michael Clausen; Ulrich Baum. *Fast Fourier transforms for symmetric groups: theory and implementation*. Math. Comp. 61 (1993), pp. 833–847, 1993.
2. Risi Kondor. *A C++ library for fast Fourier transforms on the symmetric group*.
3. Persi Diaconis; Daniel Rockmore. *Efficient computation of the Fourier transform on finite groups*. J. Amer. Math. Soc. 3 (1990), pp. 297–332, 1990.
4. Audrey Terras. *Fourier Analysis on Finite Groups and Applications*. Cambridge University Press, 1999.
5. Josh Alman; Virginia Vassilevska Williams. *A refined laser method and faster matrix multiplication*. The 32nd Annual ACM-SIAM Symposium on Discrete Algorithms, 2020.
6. Liu Cheng'ao. *Bàn về ứng dụng của DFT trong thi tin học*. Tập luận văn đội tuyển ứng viên quốc gia Trung Quốc IOI 2018, 2018.

5 Bàn về ứng dụng của một tư tưởng tham lam dựa trên so sánh trong thi tin học

Zhang Junkai, Trường Ngoại ngữ Thành Đô

Tóm tắt

Bài viết giới thiệu ứng dụng trong thi tin học của một tư tưởng tham lam dựa trên so sánh. Trước hết bài viết trình bày dạng bài toán, quá trình của phương pháp và một điều kiện cần dùng để áp dụng phương pháp này. Sau đó bài viết đưa vào một lớp bài toán trên cây có dạng tương tự, rồi đưa ra thuật toán dựa trên tư tưởng tham lam nói trên. Cuối cùng bài viết phân tích một số mô hình thường gặp trong thi tin học thỏa các điều kiện đó, cùng một phần tính chất của thuật toán.

5.1 Mở đầu

Trong thi tin học, tư tưởng tham lam là một phần cực kỳ quan trọng. Trong các lời giải tham lam có một lớp phương pháp phân tích ảnh hưởng của việc hoán đổi hai phần tử trong nghiệm lên đáp án, từ đó suy ra tính chất mà nghiệm tối ưu phải thỏa. Phương pháp này thường được gọi là *exchange arguments*. Nó đã xuất hiện trong thi tin học từ hơn mười năm trước, những năm gần đây cũng có một số ví dụ mở rộng, nhưng việc phân tích và nghiên cứu hệ thống về nó vẫn còn khá ít. Bài viết này phân tích phương pháp đó tương đối chi tiết.

Phần đầu dùng một ví dụ để dẫn nhập phương pháp, rồi phân tích một điều kiện đủ để phương pháp thu được nghiệm tối ưu đúng. Phần thứ hai giới thiệu một mở rộng cổ điển: trên dạng bài toán trước đó thêm một ràng buộc có dạng cây có gốc; sau đó đưa ra một điều kiện tương tự nhưng mạnh hơn và hai thuật toán khi bài toán thỏa điều kiện ấy. Trong cả hai phần, bài viết thảo luận một số mô hình thường gặp và phân tích một số tính chất, ứng dụng của thuật toán.

5.2 Một lớp bài toán tối ưu liên quan tới hoán vị và phương pháp exchange arguments

5.2.1 Dẫn nhập bài toán

Trong phần này ta chủ yếu xét dạng bài toán sau.

Cho n phần tử $x_1, \dots, x_n$ và một hàm F có miền xác định là các dãy gồm các phần tử này, miền giá trị là một tập có thứ tự. Với mọi hoán vị bậc n là p , hãy tìm giá trị nhỏ nhất của

$$F((x_{p_1}, x_{p_2}, \dots, x_{p_n})).$$

Ví dụ 1. Cho n cặp số nguyên dương (a_i, b_i) , hãy sắp chúng theo thứ tự tùy ý thành một dãy. Định nghĩa chi phí của dãy là tổng, trên mọi cặp trong dãy, của “ a của cặp đó nhân với tổng các b của mọi cặp đứng sau nó”. Hãy tìm chi phí nhỏ nhất.

$$n \leq 10^6, \quad 1 \leq a_i, b_i \leq 10^6.$$

Bài toán này là một trường hợp của dạng trên, với

$$F(((a_1, b_1), (a_2, b_2), \dots, (a_k, b_k))) = \sum_{1 \leq i < j \leq k} a_i b_j.$$

Rõ ràng các số thực có thể so sánh được.

Ta phân tích tính chất của một hoán vị tối ưu. Xét một hoán vị $(p_1, \dots, p_n)$ và hoán vị nhận được bằng cách đổi hai vị trí kề nhau $(i, i+1)$:

$$(q_1, \dots, q_n) = (p_1, \dots, p_{i-1}, p_{i+1}, p_i, p_{i+2}, \dots, p_n).$$

Khi đó

$$\begin{aligned} & F(((a_{p_1}, b_{p_1}), \dots, (a_{p_n}, b_{p_n}))) - F(((a_{q_1}, b_{q_1}), \dots, (a_{q_n}, b_{q_n}))) \\ &= a_{p_i} b_{p_{i+1}} - a_{p_{i+1}} b_{p_i}. \end{aligned} \tag{1}$$

Vì vậy nếu $a_{p_i}b_{p_{i+1}} - a_{p_{i+1}}b_{p_i} > 0$, sau khi đổi chỗ hai phần tử đó thì F nhỏ hơn, nên $(p_1, \dots, p_n)$ không thể là nghiệm tối ưu. Do đó chỉ các hoán vị thỏa

$$\forall 1 \leq i < n, \quad a_{p_i}b_{p_{i+1}} - a_{p_{i+1}}b_{p_i} \leq 0, \quad \text{tức} \quad \frac{a_{p_i}}{b_{p_i}} \leq \frac{a_{p_{i+1}}}{b_{p_{i+1}}},$$

mới có thể tối ưu.

Nếu một hoán vị p thỏa điều kiện đó thì với mọi $1 \leq i < j \leq n$ ta có

$$\frac{a_{p_i}}{b_{p_i}} \leq \frac{a_{p_j}}{b_{p_j}}.$$

Do đó, với mọi giá trị v , các vị trí i thỏa $a_{p_i}/b_{p_i} = v$ tạo thành một đoạn liên tiếp trong hoán vị, và các đoạn khác nhau được sắp tăng dần theo a/b . Nếu hai phần tử trong cùng một đoạn có cùng tỉ số, theo (1), đổi chỗ hai phần tử kế nhau không làm thay đổi F ; suy ra mọi hoán vị thỏa điều kiện trên có cùng giá trị F và đều tối ưu.

Vì thế với bài toán này, chỉ cần sắp mọi cặp theo a_i/b_i là thu được một cách sắp tối ưu. Độ phức tạp là thời gian sắp xếp $O(n \log n)$.

5.2.2 Phân tích bài toán và phương pháp exchange arguments

Xét cách làm trong ví dụ trên. Nó có thể được xem là xây dựng một quan hệ so sánh giữa mỗi hai phần tử x_i, x_j , cụ thể là

$$x_i < x_j \iff F((x_i, x_j)) \leq F((x_j, x_i)).$$

Trong bài đó, quan hệ so sánh được định nghĩa như vậy và bản thân bài toán thỏa một số tính chất đặc biệt, khiến việc sắp mọi phần tử theo quan hệ so sánh cho ra nghiệm tối ưu. Nhưng trong tình huống tổng quát hơn, điều này hiển nhiên không còn đúng; thực tế với một số hàm F , dạng bài toán trên hiện chỉ có thuật toán mũ. Ta phân tích các tính chất của ví dụ trước để tìm một điều kiện đủ cho thuật toán sau:

Gọi S là tập tất cả phần tử đã cho. Định nghĩa một quan hệ nhị nguyên $<$ trên S , rồi sắp mọi phần tử theo quan hệ đó.

Trong ví dụ trước, dễ thấy tồn tại các tính chất sau.

Tính chất 2.1. Với mọi $a, b \in S$, ít nhất một trong $a < b$ và $b < a$ đúng; tức quan hệ so sánh có tính toàn phần mạnh.

Tính chất 2.2. Với mọi $a, b, c \in S$, nếu $a < b$ và $b < c$ thì $a < c$; tức quan hệ có tính bắc cầu.

Tính chất 2.3. Với mọi $a, b \in S$, nếu $a < b$ thì với mọi dãy phần tử chứa $\{a, b\}$ như một dãy con liên tiếp, ta có

$$F((s_1, \dots, s_{k-1}, a, b, s_{k+2}, \dots, s_\ell)) \leq F((s_1, \dots, s_{k-1}, b, a, s_{k+2}, \dots, s_\ell)).$$

Định lý 2.1. Nếu tồn tại một quan hệ nhị nguyên $<$ trên S thỏa các Tính chất 2.1, 2.2 và 2.3, thì dùng quan hệ này trong thuật toán sắp xếp trên sẽ thu được một nghiệm tối ưu.

Chứng minh. Theo giả thiết, $<$ tạo thành một thứ tự yếu (*weak ordering*, còn gọi là quan hệ ưu tiên) trên S . Do đó, nếu $|S| = n$, tồn tại một cách xếp các phần tử thành dãy $(x_1, \dots, x_n)$ sao cho

$$\forall 1 \leq i < n, \quad x_i < x_{i+1}.$$

Với một quan hệ so sánh thỏa thứ tự yếu, thuật toán sắp xếp dựa trên so sánh có thể tìm một dãy như vậy bằng $O(n \log n)$ lần so sánh.

Bổ đề 2.1. Nếu một hoán vị $(x_1, \dots, x_n)$ của các phần tử trong S thỏa $x_i < x_{i+1}$ với mọi $1 \leq i < n$, và < thỏa Tính chất 2.3, thì không tồn tại hoán vị $(y_1, \dots, y_n)$ của S sao cho

$$F((y_1, \dots, y_n)) < F((x_1, \dots, x_n)).$$

Nói cách khác, $(x_1, \dots, x_n)$ đạt giá trị nhỏ nhất trong mọi hoán vị.

Chứng minh. Từ tính bắc cầu của <, ta có $x_i < x_j$ với mọi $i < j$. Xét một hoán vị bất kỳ $(y_1, \dots, y_n) = (x_{p_1}, \dots, x_{p_n})$. Ta biến đổi nó về $(x_1, \dots, x_n)$ bằng các lần đổi chỗ kề nhau: ở lượt i , đưa x_i về vị trí thứ i bằng cách liên tục đổi x_i với phần tử đứng ngay trước nó. Trong mỗi lần đổi, phần tử đứng trước x_i là một x_j với $j > i$, nên $x_i < x_j$. Theo Tính chất 2.3, đổi từ (x_j, x_i) sang (x_i, x_j) không làm F tăng. Sau tất cả các lần đổi, F không tăng, nên

$$F((x_1, \dots, x_n)) \leq F((x_{p_1}, \dots, x_{p_n})).$$

■

Từ bổ đề, mọi hoán vị $(x_1, \dots, x_n)$ thỏa $x_i < x_{i+1}$ đều là nghiệm tối ưu. Điều này cũng chứng minh tính đúng đắn của cách giải trong ví dụ đầu.

5.2.3 Một số ví dụ và mô hình tương ứng

Một số ví dụ. Ngoài ví dụ trên còn có các bài toán khác có dạng tương tự; dưới đây là một vài ví dụ.

Ví dụ 2. Cho n xâu $s_1, \dots, s_n$. Hãy chọn một hoán vị p của n phần tử sao cho xâu nhận được bằng cách nối $s_{p_1}, s_{p_2}, \dots, s_{p_n}$ theo thứ tự có thứ tự từ điển nhỏ nhất.

$$\sum_i |s_i| \leq 5 \times 10^5.$$

Lấy F là kết quả nối các xâu theo thứ tự; so sánh từ điển giữa các xâu là hợp lệ. Vẫn đặt $x < y$ khi và chỉ khi $F((x, y)) \leq F((y, x))$. Khi đó, với hai xâu s, t , ta có $s < t$ khi và chỉ khi $s + t$ không lớn hơn $t + s$ theo thứ tự từ điển (dấu + biểu thị phép nối xâu). Rõ ràng < thỏa Tính chất 2.1. So sánh từ điển không đổi nếu thêm cùng một xâu vào đầu hoặc cuối hai xâu cùng độ dài, nên < thỏa Tính chất 2.3. Chỉ cần chứng minh tính bắc cầu.

Bổ đề 2.2. Ký hiệu s^∞ là xâu vô hạn nhận được bằng cách lặp s vô hạn lần. Khi đó $s + t$ không lớn hơn $t + s$ theo thứ tự từ điển khi và chỉ khi s^∞ không lớn hơn t^∞ theo thứ tự từ điển.

Chứng minh bổ đề này tương đối phức tạp và ít liên quan tới phần còn lại của bài viết, nên được lược bỏ trong nguồn. Từ bổ đề, do so sánh từ điển có tính bắc cầu, quan hệ < nói trên cũng bắc cầu, vì vậy nó thỏa ba tính chất. Dùng cách làm này, với mảng hậu tố hoặc thuật toán tương tự, sau tiên xử lý có thể so sánh $s + t$ và $t + s$ trong $O(1)$. Khi đó độ phức tạp là độ phức tạp dựng cấu trúc và sắp xếp; với cách cài đặt mảng hậu tố thường gặp là $O(L \log L)$ với $L = \sum_i |s_i|$.

Ví dụ 3. Có n ổ đĩa cần định dạng. Ổ thứ i có dung lượng trước khi định dạng là a_i và sau khi định dạng là b_i . Ban đầu mỗi ổ đều chứa đầy dữ liệu; trong quá trình định dạng một ổ, cần chuyển toàn bộ dữ liệu trên ổ đó sang các ổ khác hoặc sang bộ nhớ ngoài. Dữ liệu có thể chuyển tùy ý và chia tùy ý. Hỏi cần ít nhất bao nhiêu bộ nhớ ngoài để định dạng toàn bộ ổ mà vẫn giữ được dữ liệu.

$$n \leq 10^6, \quad 0 < a_i, b_i < 10^6.$$

Xét một thứ tự định dạng $p_1, \dots, p_n$. Bài toán có thể viết thành: cho n cặp (a_i, b_i) , tìm một hoán vị tối đa hóa

$$\min_j \left\{ \sum_{i=1}^j b_{p_i} - \sum_{i=1}^j a_{p_i} \right\}.$$

Tương đương, đặt

$$F((a_1, b_1), \dots, (a_k, b_k)) = -\min_j \left\{ \sum_{i=1}^j b_i - \sum_{i=1}^j a_i \right\}$$

và tối thiểu hóa F .

Tiếp tục định nghĩa $x < y$ khi $F((x, y)) \leq F((y, x))$. Với $(a_1, b_1), (a_2, b_2)$, điều kiện này tương đương

$$\min\{-a_1, b_1 - a_1 - a_2\} \geq \min\{-a_2, b_2 - a_1 - a_2\}.$$

Để kiểm tra Tính chất 2.3 bằng cách xét đóng góp của hai phần tử kề nhau bị đổi chỗ; phần tiền tố và hậu tố ngoài hai phần tử đó giữ nguyên, còn bất đẳng thức trên đúng chính là điều kiện làm giá trị F không tăng.

Để phân tích quan hệ so sánh, xét dấu của $b_i - a_i$. Ký hiệu

$$\text{sgn}(x) = \begin{cases} 1, & x > 0, \\ 0, & x = 0, \\ -1, & x < 0. \end{cases}$$

Bổ đề 2.3. Với hai cặp $(a_1, b_1), (a_2, b_2)$:

1. nếu $\text{sgn}(b_1 - a_1) > \text{sgn}(b_2 - a_2)$ thì

$$\min\{-a_1, b_1 - a_1 - a_2\} \geq \min\{-a_2, b_2 - a_1 - a_2\};$$

2. nếu cả hai dấu đều bằng 1, bất đẳng thức trên đúng khi $a_1 \leq a_2$;
3. nếu cả hai dấu đều bằng 0, bất đẳng thức trên luôn đúng;
4. nếu cả hai dấu đều bằng -1 , bất đẳng thức trên đúng khi $b_1 \geq b_2$.

Từ bổ đề, nếu chia mọi phần tử thành ba lớp theo $\text{sgn}(b_i - a_i)$, thì trong từng lớp riêng lẻ quan hệ $<$ là một thứ tự yếu hợp lệ. Tuy nhiên giữa các lớp, quan hệ định nghĩa trực tiếp bởi $F((x, y)) \leq F((y, x))$ chưa chắc bắc cầu. Ví dụ có thể có $(2, 2) < (1, 1) < (1, 2)$ nhưng $(2, 2) \not< (1, 2)$.

Mặt khác, nếu $\text{sgn}(b_1 - a_1) > \text{sgn}(b_2 - a_2)$ thì nhất định $(a_1, b_1) < (a_2, b_2)$. Vì vậy ta định nghĩa lại $<$ như sau: $(a_1, b_1) < (a_2, b_2)$ khi và chỉ khi một trong hai điều kiện sau đúng:

1. $\text{sgn}(b_1 - a_1) > \text{sgn}(b_2 - a_2)$;
2. $\text{sgn}(b_1 - a_1) = \text{sgn}(b_2 - a_2)$ và

$$F((a_1, b_1), (a_2, b_2)) \leq F((a_2, b_2), (a_1, b_1)).$$

Để kiểm tra quan hệ này thỏa ba tính chất, nên sắp xếp theo nó giải được bài toán trong $O(n \log n)$.

Ví dụ này cho thấy vì Tính chất 2.3 chỉ cho một chiều hàm ý, không nhất thiết có thể dùng trực tiếp $F((x, y)) \leq F((y, x))$ để định nghĩa quan hệ so sánh. Nhưng từ so sánh do phép đổi chỗ gợi ra vẫn là một hướng xuất phát tốt.

Ví dụ 4. Có n hộp, hộp thứ i có trọng lượng w_i và sức chịu tải v_i ($w_i, v_i > 0$). Hãy xếp chúng thành một chồng sao cho tổng trọng lượng các hộp nằm trên mỗi hộp không vượt quá sức chịu tải của hộp đó.

$$n \leq 10^6.$$

Với một dãy hộp $((w_1, v_1), \dots, (w_n, v_n))$, cần

$$\min_j \left\{ v_j - \sum_{i < j} w_i \right\} \geq 0.$$

Nếu tối đa hóa về trái, ta được một bài toán cùng dạng với Ví dụ 3 bằng cách đặt $a_i = -v_i$ và $b_i = -v_i - w_i$. Khi đó mọi $b_i - a_i < 0$, nên một phương án tối ưu là sắp mọi hộp theo b_i giảm dần, tức theo $v_i + w_i$ tăng dần.

Tương tự, trong Ví dụ 1, nếu thay ràng buộc bằng $a_i b_i > 0$, hoặc $a_i > 0$, hoặc các điều kiện gần giống, sau khi xử lý một số phần tử đặc biệt, ta vẫn có thể so sánh (a_i, b_i) và (a_j, b_j) bằng cách tương tự so sánh $a_i b_j$ với $a_j b_i$, để quan hệ thỏa Tính chất 2.1, 2.2 và 2.3. Nhưng nếu không có ràng buộc nào và a_i, b_i là các số thực tùy ý, dễ dựng phản ví dụ cho thấy ba tính chất không thể đồng thời thỏa, nên không còn dùng được thuật toán trên.

Một số mô hình. Ba ví dụ ở trên tương ứng với ba kiểu phần tử, hàm và quan hệ so sánh khác nhau. Gọi một bộ gồm phần tử, hàm và quan hệ so sánh như vậy là một *mô hình*; các ví dụ trên cho ba mô hình:

1. Phần tử có dạng (a, b) với $a, b \in \mathbb{R}$ và $a, b \geq 0$,

$$F(((a_1, b_1), \dots, (a_n, b_n))) = \sum_{1 \leq i < j \leq n} a_i b_j,$$

so sánh F theo thứ tự thông thường trên số thực; cách so sánh phần tử là như Ví dụ 1.

2. Phần tử có dạng (a, b) với $a, b \in \mathbb{R}$,

$$F(((a_1, b_1), \dots, (a_n, b_n))) = -\min_j \left\{ \sum_{i=1}^j b_i - \sum_{i=1}^j a_i \right\},$$

so sánh theo số thực; cách so sánh phần tử cụ thể như Ví dụ 3.

3. Phần tử là xâu s , $F((s_1, \dots, s_n)) = s_1 + \dots + s_n$, so sánh theo thứ tự từ điển; hai phần tử s, t được so sánh bằng thứ tự từ điển của $s + t$ và $t + s$.

Các mô hình này, đặc biệt hai mô hình đầu, xuất hiện khá rộng trong thi tin học. Còn nhiều bài toán khác có thể chuyển thành các mô hình trên hoặc các mô hình khác thỏa ba tính chất; nguồn không triển khai thêm.

5.2.4 Một số ứng dụng khác của phương pháp

Dựa trên các tính chất và cách xử lý trong thuật toán trên, ta còn giải được nhiều bài toán hơn. Ví dụ xét lớp bài toán sau: cho n phần tử $x_1, \dots, x_n$ và một hàm F trên các dãy phần tử, thỏa ba tính chất trên. Chọn k phần tử, sắp chúng tùy ý thành $y_1, \dots, y_k$, và tối ưu hóa $F((y_1, \dots, y_k))$.

Theo thuật toán trên, ta có thể xây dựng một thứ tự yếu giữa các x_i ; giả sử $x_1 < x_2 < \dots < x_n$. Nếu các phần tử được chọn là $x_{s_1} < x_{s_2} < \dots < x_{s_k}$, thì dãy $(x_{s_1}, \dots, x_{s_k})$ là một nghiệm tối ưu cho tập đã chọn. Vì vậy luôn tồn tại nghiệm tối ưu là một dãy con của $(x_1, \dots, x_n)$. Tính chất này biến bài toán chọn một số phần tử theo thứ tự thành bài toán chọn một dãy con, trong nhiều trường hợp làm bài toán dễ hơn.

Ví dụ 5. Có n vật phẩm, vật phẩm thứ i có chi phí không âm c_i và giá trị w_i . Hai người chơi trò chơi sau:

1. Người thứ nhất chọn một vật phẩm và trả chi phí c_i . Người thứ nhất cũng có thể trực tiếp kết thúc trò chơi.
2. Người thứ hai có thể xóa vật phẩm này, khi đó trò chơi trở lại bước đầu nhưng chi phí người thứ nhất đã trả không được hoàn lại. Người thứ hai được làm thao tác này nhiều nhất k lần. Người thứ hai cũng có thể không thao tác, khi đó người thứ nhất nhận lợi ích w_i và trò chơi kết thúc.

Tổng lợi ích của người thứ nhất là lợi ích nhận được trừ tổng chi phí đã trả. Người thứ nhất muốn tối đa hóa, người thứ hai muốn tối thiểu hóa. Hãy tìm kết quả khi cả hai chơi tối ưu.

$$n < 1.5 \times 10^5, \quad k < 9.$$

Nếu người thứ nhất chọn không lấy vật phẩm, lợi ích là 0. Nếu có chiến lược tốt hơn, cuối cùng anh ta chắc chắn nhận lợi ích của một vật phẩm. Khi đó chiến lược của người thứ nhất có thể xem là chọn một dãy gồm $k + 1$ vật phẩm $(s_1, \dots, s_{k+1})$, lần thứ i chọn vật phẩm s_i . Với chiến lược cố định, lợi ích nhỏ nhất mà người thứ hai có thể buộc được là

$$\min_i \left\{ w_{s_i} - \sum_{j=1}^i c_{s_j} \right\}.$$

Vì vậy bài toán trở thành tối đa hóa giá trị này. Đây tương tự Ví dụ 4; từ cách làm của Ví dụ 4, một thứ tự tối ưu là sắp vật phẩm theo w_i tăng dần. Do đó chiến lược của người thứ nhất luôn có thể xem là chọn một dãy con sau khi sắp theo giá trị. Đặt $f_{i,j}$ là giá trị lớn nhất khi chọn j vật phẩm trong hậu tố bắt đầu từ i sau khi sắp; ta có chuyển tiếp

$$f_{i,j} = \max\{f_{i+1,j}, \min(w_i - c_i, f_{i+1,j-1} - c_i)\}.$$

Độ phức tạp là $O(nk)$; trong nguồn, do k là hằng nhỏ nên được ghi là tuyến tính theo n .

5.2.5 Tổng kết

Điểm xuất phát của thuật toán là phân tích sự thay đổi của giá trị phương án khi đổi chỗ hai phần tử kề nhau, từ đó suy ra tính chất mà phương án tối ưu cần thỏa. Vì vậy nó được gọi là *exchange argument*. Bản chất của thuật toán là tìm một quan hệ so sánh tạo thành thứ tự, sao cho mỗi khi đổi một cặp kề nhau mà trong thứ tự phần tử sau không lớn hơn phần tử trước, phương án sau đổi không kém đi; từ đó suy ra dãy thỏa thứ tự là tối ưu. Dùng trực tiếp điều kiện “đổi xong không kém đi” làm quan hệ so sánh có thể gặp vấn đề, nhưng bắt đầu từ quan hệ do phép đổi chỗ gợi ra vẫn là một phương án tốt.

Các ví dụ trên nhìn qua tưởng không liên quan, nhưng đều giải được bằng cùng một phương pháp, cho thấy thuật toán có phạm vi ứng dụng rộng. Đồng thời, tư tưởng phân tích tính chất cục bộ của nghiệm tối ưu để suy ra tính chất toàn cục cũng là một cách mạnh mẽ để phân tích bài toán.

5.3 Mở rộng bài toán lên cây có gốc và lời giải

5.3.1 Dẫn nhập bài toán

Phần này xét một lớp bài toán có dạng tương tự phần trước, nhưng thêm một ràng buộc kiểu thứ tự bộ phận liên quan tới cây có gốc. Với một hoán vị p , nói rằng x xuất hiện trước y khi tồn tại $i < j$ sao cho $p_i = x$ và $p_j = y$.

Cho n phần tử $x_1, \dots, x_n$, một hàm F có miền xác định là dãy các phần tử này và miền giá trị là một tập có thứ tự. Cho thêm một cây có gốc T gồm n đỉnh, gốc là 1; cha của đỉnh i ($2 \leq i \leq n$) là fa_i . Để mô tả gọn, ta nói “đỉnh i ứng với phần tử x_i ”.

Một hoán vị được gọi là thỏa ràng buộc của T nếu với mọi $2 \leq i \leq n$, fa_i xuất hiện trước i trong hoán vị. Hãy tìm giá trị nhỏ nhất của

$$F((x_{p_1}, x_{p_2}, \dots, x_{p_n}))$$

trên mọi hoán vị bậc n thỏa ràng buộc của T .

Ví dụ 6. Cho một cây có gốc gồm n đỉnh, gốc là 1. Mỗi đỉnh i có trọng số $w_i \in \{0, 1\}$. Cần sắp các đỉnh thành dãy $p_1, \dots, p_n$ sao cho với mọi đỉnh x , mọi tổ tiên khác chính nó của x đều xuất hiện trước x . Hãy tối thiểu hóa số nghịch thế của dãy $(w_{p_1}, w_{p_2}, \dots, w_{p_n})$, tức số cặp $1 \leq i < j \leq n$ thỏa $w_{p_i} > w_{p_j}$.

$$n < 2 \times 10^5.$$

Trong bài này, nghịch thế chỉ phát sinh khi $w_{p_i} = 1$ và $w_{p_j} = 0$. Nếu xem phần tử $w = 1$ là $(1, 0)$ trong mô hình thứ nhất, còn $w = 0$ là $(0, 1)$, bài toán có thể biểu diễn dưới dạng trên.

Nếu không có ràng buộc T , theo phần trước chỉ cần tồn tại một quan hệ so sánh giữa các phần tử thỏa ba tính chất là dùng được phương pháp exchange arguments. Nhưng khi thêm ràng buộc cây, dù F thỏa điều kiện đó, vẫn có thể không tồn tại thuật toán tốt theo hướng này. Xét mô hình sau: cho n số thực $w_1, \dots, w_n$, sắp chúng sao cho tổng k số đầu là lớn nhất. Định nghĩa $x < y$ theo so sánh số thực thông thường. Quan hệ này rõ ràng thỏa Tính chất 2.1 và 2.2, và cũng dễ chứng minh thỏa Tính chất 2.3.

Một phản ví dụ. Cho một cây có gốc gồm n đỉnh, mỗi đỉnh có trọng số. Cho số nguyên dương k , hãy chọn một khối liên thông kích thước k trên cây, chứa gốc, sao cho tổng trọng số trong khối là lớn nhất.

Bài toán tương đương chọn một hoán vị đỉnh thỏa ràng buộc của T và tối đa hóa tổng trọng số của k đỉnh đầu. Khi không có ràng buộc cây, nó tương đương mô hình vừa nêu. Bài toán có thể giải bằng quy hoạch động $O(n^2)$; nếu cây có dạng gốc có ba cây con và mỗi cây con là một đường, thì bài toán trở thành: cho ba dãy tùy ý a, b, c , tìm

$$\max_{i+j+\ell=k} \{a_i + b_j + c_\ell\},$$

nên khó giải dưới $O(n^2)$. Điều này cho thấy nếu muốn tiếp tục dùng tư tưởng tham lam dựa trên đổi chỗ, cần yêu cầu mạnh hơn về F và cần phân tích thêm.

5.3.2 Phân tích bài toán trên cây

Giả sử tồn tại một quan hệ $<$ thỏa Tính chất 2.1, 2.2 và 2.3. Ta có bổ đề sau.

Bổ đề 3.1. Giả sử các phần tử là $x_1, \dots, x_n$, gốc của cây là 1. Gọi v là đỉnh có phần tử nhỏ nhất trong $x_2, \dots, x_n$ và u là cha của v trên cây. Khi đó tồn tại một hoán vị tối ưu trong đó u và v kề nhau, tức tồn tại $i \in \{1, \dots, n-1\}$ sao cho $p_i = u, p_{i+1} = v$ hoặc $p_i = v, p_{i+1} = u$.

Chứng minh. Xét một hoán vị p bất kỳ thỏa ràng buộc của T . Giả sử $p_a = u$ và $p_b = v$. Do u là cha của v , ta có $a < b$. Vì v là phần tử nhỏ nhất ngoài gốc, mọi đỉnh nằm giữa u và v trong hoán vị đều không nhỏ hơn v . Dùng Tính chất 2.3 có thể lần lượt đổi v sang trái qua các phần tử này mà không làm F tăng. Hơn nữa, trong đoạn giữa u và v không thể có tổ tiên của u : nếu có một đỉnh như vậy, quan hệ tổ tiên trong cây sẽ mâu thuẫn với việc p thỏa ràng buộc của T . Do đó sau khi dịch v tới ngay sau u , hoán vị vẫn

thỏa ràng buộc của T . Vì với mọi hoán vị hợp lệ đều tìm được một hoán vị hợp lệ có u, v kề nhau và F không lớn hơn, tồn tại một nghiệm tối ưu có tính chất này. ■

Theo Bổ đề 3.1, nếu chỉ xét các hoán vị hợp lệ trong đó u, v kề nhau, nghiệm tối ưu thu được vẫn là nghiệm tối ưu của bài toán gốc. Hãy hợp nhất u, v trên cây thành một đỉnh đại diện cho dãy hai phần tử (x_u, x_v) . Khi đó ta được một bài toán mới trên cây có $n - 1$ đỉnh, mỗi đỉnh ứng với một dãy phần tử; chọn một hoán vị hợp lệ của các đỉnh, nối các dãy theo thứ tự hoán vị và tối thiểu hóa F trên dãy nối đó. Các dãy thu được ở bài toán mới đúng là các dãy của bài toán cũ trong đó x_u, x_v kề nhau, nên nghiệm tối ưu của bài toán mới ứng với một nghiệm tối ưu của bài toán cũ.

Bài toán mới có cùng dạng, nhưng mỗi đỉnh không còn ứng với một phần tử mà ứng với một dãy phần tử. Do đó nếu muốn tiếp tục dùng bổ đề trên, cần mở rộng quan hệ so sánh từ phần tử sang dãy phần tử.

Gọi S là tập các phần tử đã cho, và $\mathcal{T}$ là tập mọi dãy không rỗng gồm các phần tử của S . Giả sử tồn tại một quan hệ nhị nguyên $<$ trên $\mathcal{T}$ thỏa:

- **Tính chất 3.1.** Với mọi $a, b \in \mathcal{T}$, ít nhất một trong $a < b$ và $b < a$ đúng.
- **Tính chất 3.2.** Với mọi $a, b, c \in \mathcal{T}$, nếu $a < b$ và $b < c$ thì $a < c$.
- **Tính chất 3.3.** Với mọi $a, b \in \mathcal{T}$, nếu $a < b$ thì với mọi dãy phần tử s_1, s_2 ,

$$F(s_1 + a + b + s_2) \leq F(s_1 + b + a + s_2),$$

trong đó dấu $+$ là phép nối dãy.

Nếu bài toán thỏa các tính chất này, có hai thuật toán tìm một nghiệm tối ưu bằng $O(n \log n)$ lần so sánh dãy và $O(n)$ lần hợp nhất dãy. Phần sau phân tích hai thuật toán đó.

5.3.3 Hai thuật toán cho bài toán trên cây

Trong phần này giả sử các tính chất trên đều đúng. Khi đó các dãy thỏa cùng kiểu tính chất như phần tử, nên có thể thay mỗi đỉnh ban đầu ứng với một phần tử x_i bằng đỉnh ứng với dãy $s_i = (x_i)$ và chỉ xét so sánh, hợp nhất giữa các dãy.

Thuật toán 1. Tiếp tục dùng ý tưởng của Bổ đề 3.1, nhưng thay phần tử bằng dãy; bổ đề vẫn đúng. Vì vậy khi mỗi đỉnh ứng với một dãy phần tử, ta vẫn có thể hợp nhất hai đỉnh. Mỗi thao tác hợp nhất hai đỉnh trên cây và nối hai dãy tương ứng; sau $n - 1$ lần hợp nhất, cây chỉ còn một đỉnh, dãy của đỉnh đó là một nghiệm tối ưu.

Thuật toán cụ thể:

1. Lặp $n - 1$ lần.
2. Mỗi lần, tìm trong các đỉnh không phải gốc đỉnh x có dãy nhỏ nhất.
3. Nối dãy của x vào cuối dãy của cha x , rồi hợp nhất x với cha của nó trên cây.

Sau $n - 1$ thao tác, dãy ứng với gốc là một nghiệm tối ưu.

Bổ đề 3.1 bảo đảm sau mỗi lần hợp nhất, nghiệm tối ưu của bài toán mới ứng với một nghiệm tối ưu của bài toán trước, nên thuật toán đúng. Khi cài đặt, có thể dùng DSU trên cây để duy trì các đỉnh đã co, lưu thông tin của một khối liên thông tại gốc của khối đó, và dùng một cấu trúc dữ liệu hỗ trợ chèn, xóa phần tử nhỏ nhất (chẳng hạn `priority_queue` của C++) để duy trì mọi đỉnh không phải gốc. Khi đó thuật toán dùng $O(n \log n)$ lần so sánh dãy, $O(n)$ lần hợp nhất dãy; phần còn lại cũng $O(n \log n)$.

Thuật toán 2. Phân tích bài toán từ một góc nhìn khác cho ta thuật toán thứ hai. Nó dựa trên các bổ đề sau.

Bổ đề 3.2. Nói một hoán vị p thỏa quan hệ thứ tự của dãy $(v_1, \dots, v_k)$ nếu trong p , v_1 xuất hiện trước v_2 , v_2 xuất hiện trước v_3 , ..., v_{k-1} xuất hiện trước v_k .

Giả sử trong cây có gốc T , một đỉnh có các con $v_1, \dots, v_k$, và các con này đều là lá. Nếu $s_{v_1} < s_{v_2} < \dots < s_{v_k}$ thì tồn tại một hoán vị phần tử tối ưu, thỏa ràng buộc của T , đồng thời thỏa quan hệ thứ tự của dãy $(v_1, \dots, v_k)$.

Bổ đề 3.3. Giả sử một đỉnh u trong cây có hai con lá v_1, v_2 và $s_{v_1} < s_{v_2}$. Với bất kỳ hoán vị hợp lệ p , nếu $p_i = v_1, p_j = v_2$ và $j < i$, thì có thể chỉ thay đổi đoạn $p_j, p_{j+1}, \dots, p_i$ để được một hoán vị hợp lệ q trong đó v_1 xuất hiện trước v_2 và

$$F((s_{q_1} + \dots + s_{q_n})) \leq F((s_{p_1} + \dots + s_{p_n})).$$

Chứng minh. Vì $s_{v_1} < s_{v_2}$, từ Tính chất 3.1 và 3.2, khi so sánh s_{v_1} với các dãy trong đoạn giữa v_2 và v_1 , một trong hai hướng dịch chuyển tương ứng sẽ không làm F tăng theo Tính chất 3.3. Do v_1, v_2 là lá cùng cha, đưa v_1 lên trước v_2 trong đoạn này không phá vỡ ràng buộc cây. Trường hợp đối xứng xử lý tương tự. ■

Từ Bổ đề 3.3, xét hàm $g(v_1, \dots, v_k)$ là giá trị nhỏ nhất của F trên mọi hoán vị hợp lệ thỏa quan hệ thứ tự $(v_1, \dots, v_k)$. Mỗi khi trong dãy thứ tự có hai phần tử kề nhau v_i, v_{i+1} với $s_{v_i} < s_{v_{i+1}}$, đổi chúng theo chiều đúng không làm g tăng. Vì $<$ thỏa Tính chất 3.1 và 3.2, áp dụng exchange arguments của phần trước cho hàm g suy ra dãy con lá được sắp theo thứ tự không giảm là tối ưu. Do đó Bổ đề 3.2 đúng.

Bổ đề 3.4. Giả sử trong cây T , mọi con $v_1, \dots, v_k$ của đỉnh u đều là lá và $s_{v_1} < s_{v_2} < \dots < s_{v_k}$. Nếu $s_u < s_{v_i}$, thì với mọi hoán vị hợp lệ thỏa thứ tự $(v_1, \dots, v_k)$, tồn tại một hoán vị hợp lệ khác vẫn thỏa thứ tự đó, có giá trị F không lớn hơn, và trong đó u được đặt sát trước v_i theo cách tương ứng.

Chứng minh. Ý tưởng giống Bổ đề 3.3. Nếu trong một hoán vị u đứng trước v_i , dùng Tính chất 3.3 để đổi u qua các dãy nằm giữa mà không làm F tăng. Vì hoán vị ban đầu đã thỏa thứ tự của các lá, việc dịch chuyển này không phá vỡ ràng buộc cây và không phá thứ tự $(v_1, \dots, v_k)$. Trường hợp đối xứng xử lý tương tự. ■

Bổ đề 3.5. Với mỗi đỉnh u của cây T , có thể chia các phần tử trong cây con subtree $_u$ thành các dãy $c_1, \dots, c_k$ sao cho:

1. $c_1 \leq c_2 \leq \dots \leq c_k$;
2. nếu thay cây con của u bằng bài toán mới trong đó u có thêm k con lá $v_1, \dots, v_k$ lần lượt ứng với các dãy $c_1, \dots, c_k$, thì mọi nghiệm tối ưu của bài toán mới thỏa thứ tự $(v_1, \dots, v_k)$ đều ứng với một nghiệm tối ưu của bài toán ban đầu.

Chứng minh. Quy nạp trên cây. Với lá u , lấy $k = 1$ và $c_1 = (x_u)$ là hiển nhiên. Với đỉnh không phải lá, giả sử các con của u đều đã có các dãy thỏa bổ đề. Trộn các dãy của các con theo thứ tự không giảm, đồng thời giữ thứ tự nội bộ của từng con. Theo Bổ đề 3.2, tồn tại nghiệm tối ưu thỏa thứ tự đã trộn. Nếu dãy đầu tiên trong thứ tự đó nhỏ hơn dãy hiện tại của u , dùng Bổ đề 3.4 để cho hai dãy tương ứng kề nhau

rồi hợp nhất chúng. Lặp thao tác này cho đến khi dãy đầu tiên không nhỏ hơn dãy của u , hoặc mọi lá đã bị hợp nhất. Khi đó các dãy còn lại, cùng dãy mới của u , thỏa hai điều kiện của bổ đề. ■

Từ cách xây dựng trong chứng minh Bổ đề 3.5, ta có thuật toán thứ hai. Với mỗi đỉnh u , tính dãy S_u gồm các dãy c_i của Bổ đề 3.5 theo thứ tự. Với một đỉnh u , thực hiện:

1. Trước hết lấy tất cả S_v của các con v của u , rồi trộn chúng theo quan hệ so sánh dãy để được S_u .
2. Đặt $C = (x_u)$.
3. Lặp: lấy phần tử đầu c của S_u . Nếu S_u rỗng hoặc $c \not\prec C$ thì dừng; nếu không, xóa c khỏi S_u và nối c vào cuối C .
4. Sau khi dừng, thêm C vào đầu S_u .

Duyệt DFS từ dưới lên để tính mọi S_u . Sau khi có S_1 của gốc, nối mọi dãy trong S_1 theo thứ tự là một nghiệm tối ưu. Thuật toán cần $O(n)$ lần trộn hai danh sách S , $O(n)$ lần hỏi/xóa phần tử đầu của một S và $O(n)$ lần hợp nhất dãy. Nếu dùng cây cân bằng hoặc cấu trúc tương tự để duy trì S , thuật toán dùng $O(n \log n)$ lần so sánh dãy, cùng bậc với thuật toán 1.

5.3.4 Phân tích các mô hình cụ thể

Với các bài toán mà tồn tại quan hệ so sánh thỏa Tính chất 3.1, 3.2 và 3.3, hai thuật toán trên đều giải được bằng $O(n \log n)$ lần so sánh dãy và $O(n)$ lần hợp nhất dãy. Sau đây xét ba mô hình đã nêu ở phần trước.

Mô hình thứ nhất. Trong mô hình thứ nhất, phần tử có dạng (a, b) với $a, b \in \mathbb{R}$, $a, b \geq 0$, và

$$F(((a_1, b_1), \dots, (a_n, b_n))) = \sum_{1 \leq i < j \leq n} a_i b_j.$$

Với hai dãy $c_1 = ((a_1, b_1), \dots, (a_n, b_n))$ và $c_2 = ((a'_1, b'_1), \dots, (a'_m, b'_m))$, ta có

$$F(c_1 + c_2) - F(c_2 + c_1) = \left(\sum_{i=1}^n a_i \right) \left(\sum_{i=1}^m b'_i \right) - \left(\sum_{i=1}^m a'_i \right) \left(\sum_{i=1}^n b_i \right).$$

Vì vậy một dãy có thể xem như phần tử tương đương

$$\left(\sum_i a_i, \sum_i b_i \right).$$

Dùng cách so sánh của mô hình thứ nhất trên hai phần tử tương đương này để so sánh hai dãy. Dễ thấy quan hệ đó thỏa Tính chất 3.1, 3.2 và 3.3.

Mô hình thứ hai. Trong mô hình thứ hai, phần tử có dạng (a_i, b_i) với $a_i, b_i \in \mathbb{R}$, và

$$F(((a_1, b_1), \dots, (a_n, b_n))) = - \min_j \left\{ \sum_{i=1}^j b_i - \sum_{i=1}^j a_i \right\}.$$

Với hai phần tử liên tiếp $(a_1, b_1), (a_2, b_2)$, có thể hợp nhất chúng thành một phần tử tương đương

$$(- \min\{-a_1, b_1 - a_1 - a_2\}, \quad b_1 + b_2 - a_1 - a_2 - \min\{-a_1, b_1 - a_1 - a_2\}),$$

mà giá trị F đối với mọi ngữ cảnh bên ngoài không đổi. Quy nạp cho thấy mọi dãy đều có thể hợp nhất thành một phần tử tương đương khi tính F . Do đó trong Tính chất 3.3 có thể thay các dãy ở hai vế bằng phần tử tương đương của chúng, rồi dùng quan hệ so sánh phần tử của mô hình thứ hai. Vì quan hệ phần tử thỏa Tính chất 2.1, 2.2 và 2.3, quan hệ dãy thu được thỏa Tính chất 3.1, 3.2 và 3.3.

Mô hình thứ ba. Với mô hình này, nối một số vào đầu dây vẫn thu được một dây, nên các tính chất trên hiển nhiên đúng. Chi tiết về cài đặt nối và so sánh dây phức tạp hơn, nguồn không triển khai.

Ngược lại, với phần ví dụ ở đầu phần này, không tồn tại quan hệ $<$ thỏa điều kiện trên. Chẳng hạn lấy $k = 2$, khi đó có thể có $F((3, 3)) > F((4, 1))$ nhưng $F((4, 3, 2)) < F((4, 4, 1))$. Theo Tính chất 3.1, hai dây $(3, 3)$ và $(4, 1)$ phải có một dây không lớn hơn dây kia, nhưng cả hai khả năng đều mâu thuẫn với Tính chất 3.3. Vì vậy điều kiện nói trên chắc chắn không thỏa, và bài toán đó không giải được bằng thuật toán này.

5.3.5 Tính chất và ứng dụng của thuật toán

Hai mô hình đầu có một tính chất tốt: khi hợp nhất hai dây c_1, c_2 , chỉ cần biết $O(1)$ thông tin của mỗi dây là có thể tính thông tin của dây hợp nhất và $F(c_1 + c_2)$. Với mô hình thứ nhất, chỉ cần biết $\sum a_i$, $\sum b_i$ và $F(c)$; với mô hình thứ hai, chỉ cần biết phần tử tương đương (a, b) . Vì thế đối với các bài toán thuộc hai mô hình này, hai thuật toán trên có thể tính đáp án trong $O(n \log n)$. Mô hình thứ nhất ứng với Ví dụ 6; bài toán sau ứng với mô hình thứ hai.

Ví dụ 7. Cho một cây n đỉnh. Mỗi đỉnh trừ đỉnh 1 có trọng số v_i . Một người ban đầu ở đỉnh 1 và có một giá trị hp , ban đầu bằng 0. Người này có thể di chuyển dọc theo các cạnh của cây; khi lần đầu tới đỉnh i , hp được cộng thêm v_i . Nếu $hp < 0$ thì người đó thất bại. Hỏi có thể tới được một đỉnh cho trước t mà không thất bại hay không.

$$n < 10^6, \quad |v_i| < 10^6.$$

Thêm một lá nối với 1 có trọng số rất lớn. Nếu đi được tới t , sau khi đi tới lá này ta chắc chắn có thể đi qua mọi đỉnh còn lại; vì vậy bài toán trở thành có thể đi qua mọi đỉnh mà không thất bại hay không. Nếu ghi lại dãy đỉnh theo lần đầu tiên đi qua, ta được một hoán vị thỏa ràng buộc của cây T , và quá trình thay đổi của hp chính là đi qua các phần tử theo hoán vị đó.

Bài toán trở thành tìm một hoán vị hợp lệ sao cho mọi thời điểm $hp \geq 0$. Tương đương, tối đa hóa giá trị nhỏ nhất của hp trong quá trình. Đây là mô hình thứ hai: phần tử $x \geq 0$ ứng với $(0, x)$, còn $x < 0$ ứng với $(-x, 0)$. Dùng thuật toán trên cho độ phức tạp $O(n \log n)$.

Tìm một nghiệm tối ưu. Như đã nói, trong hai mô hình đầu chỉ cần duy trì một phần thông tin của dây để so sánh và tính đáp án. Nếu muốn khôi phục phương án, trong cả hai thuật toán đều có n lần hợp nhất dây, mỗi lần dạng “nối dây t vào cuối dây s ”. Ghi lại các thao tác hợp nhất đó; sau khi thuật toán kết thúc, có thể khôi phục dãy phương án trong $O(n)$.

Tìm đáp án cho bài toán con của mỗi cây con. Xét dạng bài toán sau: với mỗi đỉnh x , bài toán con của cây con gốc x là lấy các đỉnh trong cây con của x làm cây mới T' , lấy x làm gốc, và giữ nguyên dạng bài toán. Hãy tìm đáp án của bài toán con với mọi đỉnh x .

Ví dụ 8. Cho một cây có gốc gồm n đỉnh, gốc là 1, mỗi đỉnh có trọng số v_i . Ta có một số quân cờ và được thực hiện hai loại thao tác:

1. Chọn một đỉnh đang có quân cờ và thu lại toàn bộ quân cờ trên đỉnh đó.
2. Chọn một đỉnh x chưa có quân cờ, đồng thời mọi con của x đều đang có quân cờ, rồi đặt v_x quân cờ lên x .

Với mỗi đỉnh, hỏi nếu cần đặt quân cờ lên đỉnh đó thì ban đầu cần ít nhất bao nhiêu quân cờ.

$$n < 2 \times 10^5, \quad 1 < v_i < 10^9.$$

Phân tích bài toán cho hai tính chất:

1. Nếu cần đặt quân cờ lên đỉnh i , quá trình chỉ đặt quân cờ trong cây con của i và đặt đúng một lần lên mỗi đỉnh trong cây con đó.
2. Tồn tại một cách thao tác tối ưu sao cho sau khi đặt quân cờ lên một đỉnh x , thao tác tiếp theo là thu lại toàn bộ quân cờ ở các con của x .

Gọi $p_1, \dots, p_k$ là dãy các đỉnh được đặt quân cờ. Với bài toán của đỉnh i , tồn tại nghiệm tối ưu mà dãy này là một hoán vị của cây con i . Đặt s_i là tổng v của các con của i ; sau khi đặt quân cờ lên p_j , số quân đang dùng là

$$\sum_{\ell=1}^j v_{p_\ell} - \sum_{\ell < j} s_{p_\ell}.$$

Vì vậy bài toán tương đương tối thiểu hóa giá trị lớn nhất của biểu thức này. Sau khi đảo thứ tự dãy, dạng hàm thu được tương tự mô hình thứ hai, và hướng ràng buộc trên cây cũng trở về dạng cha xuất hiện trước con. Do đó bài toán được chuyển thành tìm đáp án bài toán con của mỗi cây con.

Với thuật toán 2, Bổ đề 3.5 cho ngay tính chất: nếu S_x là dãy các dãy được xây dựng cho đỉnh x , thì nối các phần tử trong S_x theo thứ tự là một nghiệm tối ưu của bài toán con gốc x . Do đó, nếu dùng cây cân bằng hoặc cấu trúc tương tự để duy trì S_x , đồng thời có thể duy trì nghiệm tối ưu của cây con gốc x ; độ phức tạp cùng bậc với thuật toán 2.

Thuật toán 1 cũng giải được yêu cầu này, nhưng cần phân tích thêm.

Bổ đề 3.6. Với một đỉnh u bất kỳ trong cây có gốc T , xét một tập con S của các con của u . Gọi $U(u, S)$ là tập gồm u và mọi đỉnh thuộc các cây con của các đỉnh trong S . Với bất kỳ hoán vị tối ưu p thu được bằng thuật toán 1 trên T , tồn tại một hoán vị q thu được bằng thuật toán 1 trên cây cảm sinh bởi $U(u, S)$, lấy u làm gốc, sao cho q giữ thứ tự tương ứng của các đỉnh trong p khi chỉ xét $U(u, S)$.

Chứng minh. Quy nạp theo số đỉnh của T . Xét thao tác hợp nhất đầu tiên trong thuật toán 1. Nếu thao tác không liên quan tới $U(u, S)$, cấu trúc phần này không đổi và dùng giả thiết quy nạp. Nếu thao tác chỉ ảnh hưởng tới gốc u của phần đang xét, quá trình thuật toán 1 trong phần đó không phụ thuộc vào dãy đang nằm tại gốc, nên kết luận vẫn đúng. Nếu thao tác hợp nhất hai đỉnh đều thuộc $U(u, S)$, thì đỉnh được hợp nhất cũng là một đỉnh có dãy nhỏ nhất trong bài toán cảm sinh bởi $U(u, S)$, nên có thể chọn cùng thao tác đầu tiên trong thuật toán 1 của bài toán con; sau đó áp dụng quy nạp. ■

Từ Bổ đề 3.6, với mọi hoán vị tối ưu p do thuật toán 1 sinh ra và mọi đỉnh x , nếu trong p chỉ giữ các đỉnh thuộc cây con của x , ta thu được một nghiệm tối ưu của bài toán con cây con x theo thuật toán 1. Vì vậy có thể dùng cấu trúc dữ liệu hỗ trợ hợp nhất, duyệt DFS từ dưới lên để duy trì một nghiệm tối ưu cho mỗi đỉnh. Một cách cài đặt là dùng cây đoạn động và hợp nhất cây đoạn; độ phức tạp cùng bậc với thuật toán 1.

Bổ đề 3.5 cũng cho thấy hoán vị tối ưu do thuật toán 2 sinh ra có tính chất tương tự. Tuy nhiên nếu lấy một hoán vị tối ưu bất kỳ không do hai thuật toán trên sinh ra, để dựng phản ví dụ cho tính chất này; vì vậy kết luận chỉ được bảo đảm với nghiệm do hai thuật toán trong bài tạo ra.

Một số ứng dụng khác. Cuối cùng xét một bài toán liên quan tới cách làm của thuật toán 2.

Ví dụ 9. Có n đồng đá xếp thành một hàng, đồng thứ i có a_i viên đá. Cần hợp nhất mọi đồng thành một đồng; mỗi lần chọn hai đồng kề nhau và hợp nhất chúng, chi phí là tổng số đá của hai đồng. Có thêm ràng buộc: trong hai đồng được chọn ở mỗi lần, phải có đồng chứa đồng đá ban đầu thứ x . Hãy tìm chi phí nhỏ nhất.

$$n < 2 \times 10^5, \quad 1 < a_i < 10^9.$$

Với một vị trí x cố định, quá trình có thể xem là mỗi lần nhập một đồng vào đồng thứ x . Nếu lần thứ $i - 1$ nhập đồng p_i ($p_1 = x$), ta thu được một hoán vị. Ràng buộc của hoán vị tương đương với ràng buộc của cây là đường $1 - 2 - \dots - n$ lấy x làm gốc. Tổng chi phí chỉ phụ thuộc vào hoán vị; sau khi trừ một hằng số, nó có dạng

$$\sum_{1 \leq i < j \leq n} a_{p_j},$$

tức mô hình thứ nhất với phần tử $(a_i, 1)$. Vì vậy một bài toán cho x cố định giải được trong $O(n \log n)$.

Nếu dùng thuật toán 2, có thể xử lý mọi x liên tiếp. Gọi L_x là dãy các dãy nhận được từ đoạn $1 - 2 - \dots - (x - 1)$ khi lấy $x - 1$ làm gốc, và R_x tương tự cho đoạn $(x + 1) - \dots - n$ khi lấy $x + 1$ làm gốc. Theo Bổ đề 3.2 và 3.4, nếu trộn L_x và R_x theo quan hệ so sánh thành S_x , rồi nối mọi dãy trong S_x và thêm x vào đầu, ta được một nghiệm tối ưu cho gốc x .

Khi x tăng dần từ 1 tới n , các thay đổi của L_x và R_x chỉ gồm $O(n)$ lần chèn hoặc xóa ở đầu và truy vấn dãy đầu. Duy trì kết quả trộn hiện tại bằng cây cân bằng: khi xóa dãy đầu thì xóa trực tiếp; khi chèn một dãy vào đầu L hoặc R , nhờ cách xây dựng, dãy mới không lớn hơn mọi dãy hiện có ở phía đó, nên có thể chèn vào vị trí thích hợp trong kết quả trộn mà vẫn giữ cả thứ tự toàn cục lẫn thứ tự nội bộ. Đồng thời duy trì giá trị đáp án trên cây cân bằng. Vì so sánh, nối dãy và hợp nhất thông tin đáp án đều duy trì được trong $O(1)$ cho mô hình này, tổng độ phức tạp là $O(n \log n)$.

Trong bài này chỉ cần đáp án. Nếu hai dãy a, b thỏa $a < b$ và $b < a$, đổi chỗ hai dãy này không làm thay đổi F , nên khi chèn có thể sắp các dãy tương đương theo thứ tự tùy ý mà đáp án vẫn tối ưu. Tuy nhiên nếu chèn tùy ý, phương án đang duy trì có thể không còn thỏa ràng buộc cây, dù giá trị đáp án vẫn đúng.

5.3.6 Tổng kết

Lớp bài toán này có cấu trúc tương tự phần trước, nhưng thêm ràng buộc lên hoán vị có dạng cây có gốc. Về điều kiện cần thỏa và cách giải, hai lớp bài toán có liên hệ rất mạnh. Tuy nhiên các ví dụ cho thấy chỉ thỏa yêu cầu của phần trước không đủ để giải bài toán có ràng buộc cây. Bài viết tăng cường điều kiện: thay vì chỉ so sánh phần tử, cần so sánh cả dãy phần tử. Trên cơ sở đó, bài viết đưa ra hai thuật toán từ hai góc nhìn khác nhau. Thuật toán thứ nhất bắt đầu từ tính chất cục bộ của một đỉnh, liên tục hợp nhất đỉnh có dãy nhỏ nhất với cha của nó để thu nhỏ bài toán. Thuật toán thứ hai trước hết giải các cây con của con, rồi dùng các tính chất đã chứng minh để hợp nhất lời giải cây con thành lời giải của cây con hiện tại. Hai thuật toán có cùng độ phức tạp. Từ quá trình của hai thuật toán có thể suy ra thêm nhiều ứng dụng; bài viết chỉ phân tích một số ứng dụng mà tác giả biết.

5.4 Lời kết và triển vọng

Trong thi tin học, tư tưởng tham lam có phạm vi ứng dụng rất rộng và còn nhiều vấn đề đáng nghiên cứu sâu. Các thuật toán liên quan trong bài viết đã nhiều lần xuất hiện trong thi tin học, nhưng việc phân tích chi tiết chúng không đơn giản. Do trình độ tác giả có hạn, vẫn còn nhiều câu hỏi liên quan tới lớp thuật toán này chưa có câu trả lời tốt, chẳng hạn:

1. Hai phần trong bài đều đặt ra ba tính chất cho quan hệ so sánh. Có tồn tại các tính chất yếu hơn hoặc đơn giản hơn, nhưng vẫn bảo đảm thuật toán trong bài thu được nghiệm tối ưu hay không?

2. Ngoài ba mô hình và các biến thể đã nêu, có tồn tại nhiều mô hình khác thỏa các tính chất trên hay không? Có mô tả bản chất hơn cho các mô hình đó hay không?

Tác giả hy vọng bài viết có thể gợi mở thêm nghiên cứu, để các bạn đọc quan tâm tìm được nhiều cách làm và ứng dụng thú vị hơn cho lớp bài toán này.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn huấn luyện viên đội tuyển quốc gia Yang Ziliang đã hướng dẫn; cảm ơn cha mẹ đã nuôi dưỡng và giáo dục; cảm ơn nhà trường đã bồi dưỡng; cảm ơn huấn luyện viên Li Xiang và các huấn luyện viên của Trường Ngoại ngữ Thành Đô đã chỉ bảo nhiều mặt; cảm ơn bạn Dai Jiangqi đã trao đổi và gợi mở; cảm ơn Liu Yiping, Jin Tian, Song Jiaying và nhiều bạn khác đã đọc kiểm tra bản thảo; cảm ơn anh Wen Kairui đã động viên và giúp đỡ tác giả trong quá trình học thi tin học.

Tài liệu tham khảo

1. Wikipedia contributors. *Weak Orderings*. https://en.wikipedia.org/wiki/Weak_ordering.
2. Errichto. *Lecture #3 – Exchange arguments (sorting with dp)*. <https://codeforces.com/blog/entry/63533>.
3. AtCoder Grand Contest 023 Editorial. <https://atcoder.jp/contests/agc023/editorial>.
4. CERC 2013: Presentation of solutions. <https://cerc.tcs.uj.edu.pl/2013/data/cerc2013solutions.pdf>.

6 Chủ đề chính

Từ mục lục đã đọc được trên ảnh render, tập năm 2022 bao phủ các nhóm chủ đề sau:

- duy trì thông tin đường đi trên DAG, DFT và các biến thể tổng quát;
- tư tưởng tham lam dựa trên so sánh và thuật toán xấp xỉ;
- đồ thị phẳng, tô màu đồ thị và các bài toán tương tác dạng tìm tham số;
- tối ưu không gian, ma trận Monge và phân tích bảng băm;
- đại số tuyến tính, lý thuyết Lyndon, tổ hợp chuỗi và mô phỏng luồng chi phí.

7 Ghi chú về trích xuất

pdftotext không trích xuất được văn bản có nghĩa từ tập 2022; kết quả kiểm tra các trang đầu chỉ gồm dấu sang trang. Mục lục và bài đầu tiên trong bản dịch này được đọc bằng cách render bằng pdftoppm rồi OCR bằng tesseract với ngôn ngữ chi_sim+eng. Một số công thức ở bài đầu tiên, cụ thể các mục 2.1, 2.3, 3.2 và 3.3, bị nhiễu trên ảnh/OCR; các chỗ đó được đánh dấu bằng ghi chú OCR tiếng Việt thay vì tự suy diễn công thức. Bài thứ hai được OCR từ các trang vật lý 17–31 của PDF, tương ứng trang mục lục 13–27; các khối công thức quan trọng được đối chiếu lại trên ảnh render. Bài thứ ba được OCR từ các trang vật lý 32–52 của PDF, tương ứng trang mục lục 28–48. Bài này không có hình; các công thức chính được đối chiếu lại trên ảnh render. Một số chứng minh dài trong các mục 3.3–3.5 bị OCR nhiễu

nặng, nên bản dịch giữ phát biểu toán học, cấu trúc lập luận và ý chứng minh thay vì cố chép lại từng dòng biến đổi không đọc chắc được.